Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Eléctrica
IEE3615 – Aprendizaje Reforzado Para Control de Sistemas Dinámicos

Informe Tarea 2

Nombre: Felipe Robino Schütze

Pregunta 1

Pregunta 2

(a)

Como se menciona en el enunciado, DWPT consiste en la transmisión energética mediante bobinas instaladas en la calle hacia vehículos eléctricos que circulan por ella. Esto tiene el fin de reducir el tamaño de las baterías de los EVs sin comprometer el rango de ellos. En este balance de costos sale a la luz qué tanto más eficiente es proveer las calles y EVs de las bobinas, inversores, y energía para que el sistema funcione. Debido a esto, las aplicaciones más convincentes de DWPT son aquellas con una escala de implementación más concisa y clara. Principalmente, en electromovilidad de buses eléctricos urbanos (con rutas conocidas y definidas), o en tramos de carreteras que carguen los vehículos que pasan por ella. El fin de las aplicaciones en general es reducir las necesidades de batería de los EVs, reduciendo costos. Otras aplicaciones son en bodegas o puertos, donde AGVs (Autonomous Guided Vehicles) tienen rutas más concisas y definidas, y se cargan mediante DWPT.

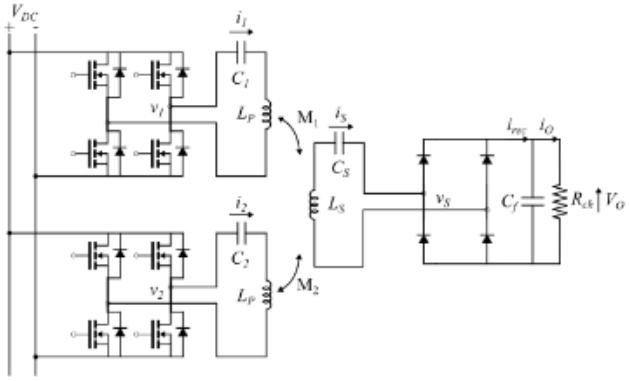


Figura 1: Circuito equivalente de esquema DWPT.

Si vemos el circuito equivalente de DWPT (Figura 1), podemos notar que el circuito primario consiste de una bobina con un capacitor (circuito LC). La impedancia de este circuito será su autoimpedancia más la impedancia reflejada del secundario:

$$\begin{aligned} Z_{in} &= Z_P + Z_{ref} \\ &= \left[R + j\left(\omega L_P - \frac{1}{\omega C_P}\right)\right] + \frac{(\omega M)^2}{Z_s} \end{aligned}$$

Si obviamos la impedancia del secundario, conviene operar cerca de la resonancia del primario puesto que minimiza el término Z_P, con $\omega_0 = \frac{1}{\sqrt{L_P C_P}}$:

$$Z_P(\omega_0) = R + j\left(\omega_0 L_P - \frac{1}{\omega_0 C_P}\right) = R$$

Esto implica que la transferencia de potencia será máxima para esta frecuencia de conmutación (las corrientes de reactancia son nulas). La presencia del secundario (si asumimios que es LC) hace que la

impedancia total del circuito sea:

$$\begin{aligned} Z_{in} &= Z_P + Z_{ref} \\ &= \left[R + j\left(\omega L_P - \frac{1}{\omega C_P}\right)\right] + \frac{(\omega M)^2}{Z_s} \\ &= \left[R + j\left(\omega L_P - \frac{1}{\omega C_P}\right)\right] + \frac{(\omega M)^2}{j\left(\omega L_S - \frac{1}{\omega C_S}\right)} \\ &= R + j\left[\omega L_P - \frac{1}{\omega C_P} - \left(\frac{(\omega M)^2 \omega C_S}{\omega^2 L_S C_S - 1}\right)\right] \end{aligned}$$

La nueva frecuencia de resonancia es aquella que minimiza el término $\omega L_P - \frac{1}{\omega C_P} - \left(\frac{(\omega M)^2 \omega C_S}{\omega^2 L_S C_S - 1}\right)$, en particular:

$$\omega_0' = \sqrt{2}\sqrt{\frac{1}{\sqrt{(C_p L_p - C_s L_s)^2 + 4 C_p C_s M^2} + C_p L_p + C_s L_s}} \approx \frac{1}{\sqrt{C_p L_{eq}}}$$

Con $L_{eq} = L_p - \frac{M^2}{L_S}$

Conviene ir activando cada bobina a medida que el auto se acerca a ella, es decir, cuando el acomplamiento con esa bobina en particular sea máximo. La frecuencia de conmutación de cada bobina primaria también debería ir ajustándose al cambio de resonancia recién descrito, que depende de M. Esto haría que se minimicen las pérdidas energéticas del sistema. La estrategia que esperaría que el controlador empleara, es la de ir encendiendo las bobinas más cercanas al auto (siguiendo su trayectoria), y a medida que avanza a través de cada una, que la frecuencia de conmutación de cada bobina y potencia se fueran ajustando a la resonancia y acoplamiento M_i de cada bobina respecto al auto.

(b)

Al correr `p2_main.py` se guardan todos los barridos y gráficos pedidos en el directorio `\results`.

- **Perfil de acoplamiento magnético:**

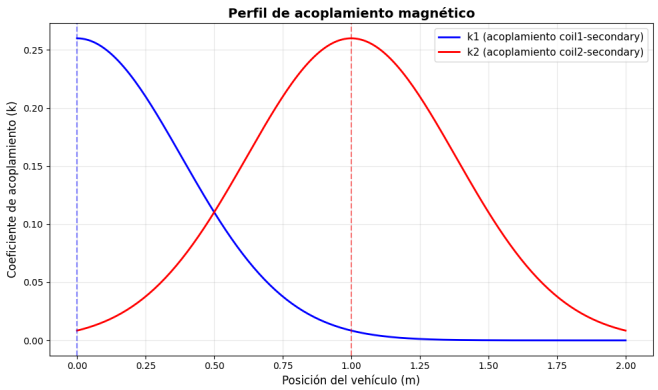


Figura 2: Perfil de acoplamiento magnético (k_1, k_2 en función de x).

Analizando la Figura 2, podemos ver que existe un solapamiento significativo entre los acoplamientos de las bobinas para distintas posiciones del vehículo. Esto es indicativo que existe una región donde ambas bobinas deben trabajar juntas para obtener una mayor potencia transferida. Para la bobina

1 se debe apagar antes de 1.35 m (ya que la potencia es nula más allá de esto), pero el punto exacto y proporción a la que se debe ir apagando esta bobina y encendiendo la segunda es más difícil de interpretar sólo teniendo este gráfico (habría que ver la potencia efectiva transmitida y eficiencia para determinarlo exactamente). Sin embargo, podemos afirmar que la bobina 1 debe estar encendida a su máxima potencia en $x = 0$, bajando la proporción en la que se usa respecto a la bobina 2 paulatinamente, siendo igual esta proporción en $x = 0.50$ ($k_1 = k_2$). Luego la bobina 2 va subiendo hasta su máximo valor en $x = 1$ y siguiendo el ciclo con la siguiente bobina.

- **Potencia transferida vs. frecuencia de conmutación:**

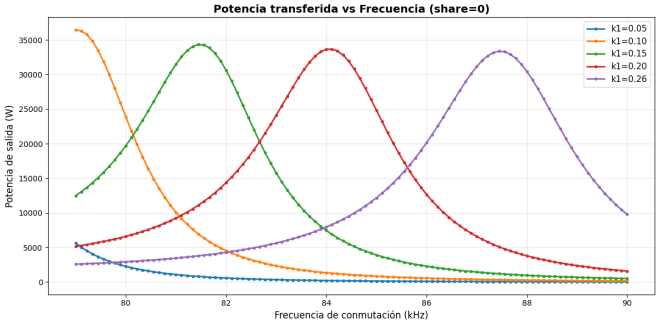


Figura 3: Potencia transferida P_{out} vs. frecuencia de conmutación f para distintos valores de acoplamiento k_1.

En la Figura 3 podemos apreciar cómo para distintos valores de acoplamiento k_1, existen distintos peaks de potencia de salida, todas bajo distintas frecuencias de conmutación. Las frecuencias óptimas para cada acoplamiento son directamente proporcionales a la magnitud del acoplamiento (a mayor k_1 mayor f_0), y además existe un leve decrecimiento de la potencia máxima de salida alcanzada respecto al acoplamiento. Esto se puede explicar con la ecuación encontrada en (a) que determina $\omega_o \approx \frac{1}{\sqrt{C_p L_{eq}}}$, con $L_{eq} = L_p - \frac{M^2}{L_S}$. Como M está determinado por k_1, la frecuencia óptima también lo estará y la potencia a dicha frecuencia también.

- **Eficiencia vs. frecuencia de conmutación:** En la Figura 4 podemos apreciar un caso consistente

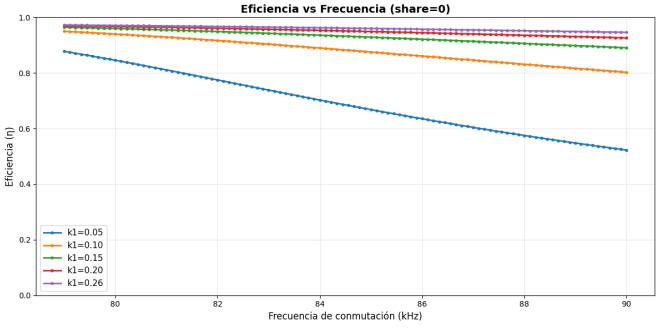


Figura 4: Eficiencia η vs. frecuencia de conmutación f para distintos valores de acoplamiento k_1.

al de la potencia con la eficiencia. Se ve que en general la eficiencia tiende a disminuir con la frecuencia de operación, y qué tan rápido baja y empieza dicha eficiencia se ve determinado por el acoplamiento. A menor acoplamiento, menor eficiencia que decae además más rápidamente con la frecuencia. Esto es consistente con lo visto en la Figura 3, puesto que al desplazar la frecuencia

óptima hacia frecuencias mayores, se opera bajo menores eficiencias, disminuyendo la potencia total de salida levemente.

- **Portencia transferida vs. desfase:**

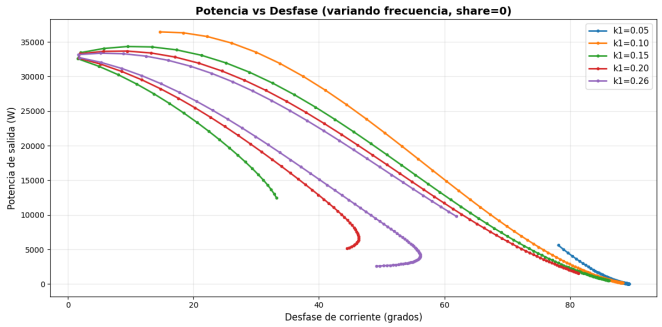


Figura 5: Potencia transferida P_{out} vs. desfase $phase1\_deg$ para distintos valores de acoplamiento k_1.

En la Figura 5 se pueda apreciar cómo para todos los perfiles de acoplamiento, mientras mayor el desfase menor es la potencia transferida. Dado esto, sería deseable operar cerca del desfase nulo ($0°$) para lograr buenas potencias de transmisión. El ángulo de desfase es útil puesto que cercano a la resonancia el desfase tiende a ser nulo, es decir, medir qué tan pequeño es el desfase es una buena forma de medir qué tan cercano a la resonancia estoy operando.

- **Potencia transferida vs. distribución de potencia entre bobinas:**

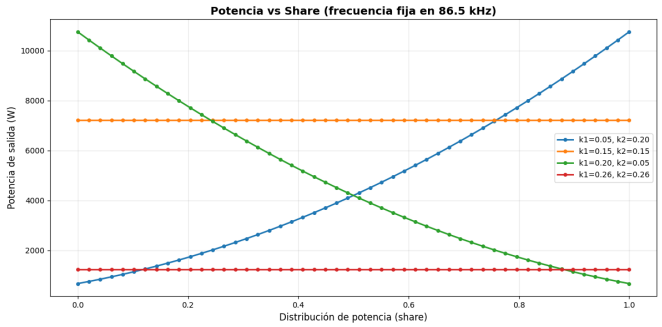


Figura 6: Potencia transferida P_{out} vs. distribución de potencia entre bobinas $share$ para distintos pares de acoplamiento (k_1, k_2).

Analizando la Figura 6, podemos apreciar 3 casos. El primero es cuando $k_1 = k_2$, acoplamientos iguales entre las bobinas, caso en el cual la potencia transferida es independiente de share. Esto se debe a que es indiferente a cuál de las bobinas transmitimos la potencia (ambas transmiten la misma a la salida), por lo que al repartirla entre cualquiera de las dos bajo cualquier proporción es igual. El segundo caso es cuando $k_1 > k_2$. En este caso es preferible usar mucho más la bobina 1 que la 2 (dado que tiene mayor acoplamiento), por lo que usar shares mayores resulta en potencia perdida (no transmitida) en la segunda bobina y una menor potencia de salida. El tercer caso es el opuesto $k_1 < k_2$, caso en el que conviene usar shares mayores para usar la bobina dominante k_2, análogamente al caso contrario.

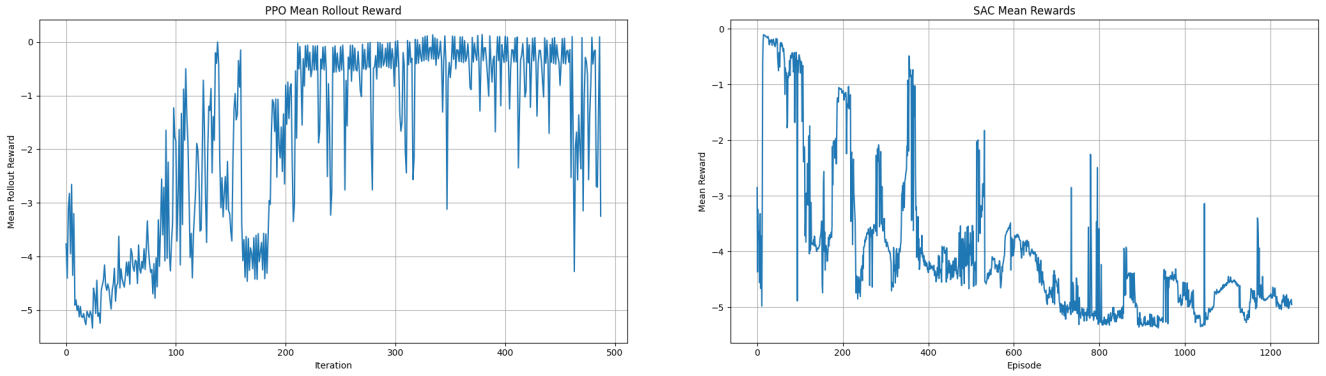
(c)

Para el caso de SAC, se utilizó el archivo `sac.py` con modificaciones mínimas (formato retorno de función `train` y tracking de rewards) que fue proporcionado, y se implementó al final de `p2_main.py`. En el caso de PPO, se utilizó como base el repositorio CleanRL[1], específicamente el archivo `continous_action_ppo`. El archivo original contenía muchas variables y funciones destinadas a entrenar agentes para simuladores de atari o cosas más complejas, pero la estructura de ppo sirvió tras podar dichas funciones inecesarias para aplicarlo al environment DWPT. La función de rewards se implementó directamente en `DWPT_rl.py`, en el caso de acceso a las variables del secundario (`full`), tiene la siguiente forma:

```python
def compute_reward_full(self):
    power_term = self.pout / self.p.p_rated
    efficiency_term = self.eta
    phase_penalty = phase_band_penalty(self.phase_active_mag_deg)

    reward = (
        1.0 * power_term
        + 0.15 * efficiency_term
        - 0.25 * phase_penalty
    )

    return float(reward)
```

La idea de esto es que el controlador aprenda a transmitir la mayor potencia posible (power_term) al auto, manteniendo buena eficiencia (efficiency_term) y cercanía a la frecuencia de conmutación óptima (phase_penalty). Al incorporar la función directamente en el environment (parte de step), logramos comparar ambos algoritmos de manera justa, usando exactamente la misma función de rewards, seed, y número de pasos. Así, al correr `p2_main.py` y seleccionar la opción `full`, se entrenan ambos algoritmos y rolloutean las políticas.

Para visualizar el entrenamiento, como los algoritmos usan losses y arquitecturas fundamentalmente distintas, se pueden ver los rewards medios, los retornos por episodio, y la entropía de PPO vs alpha de SAC (exploración).

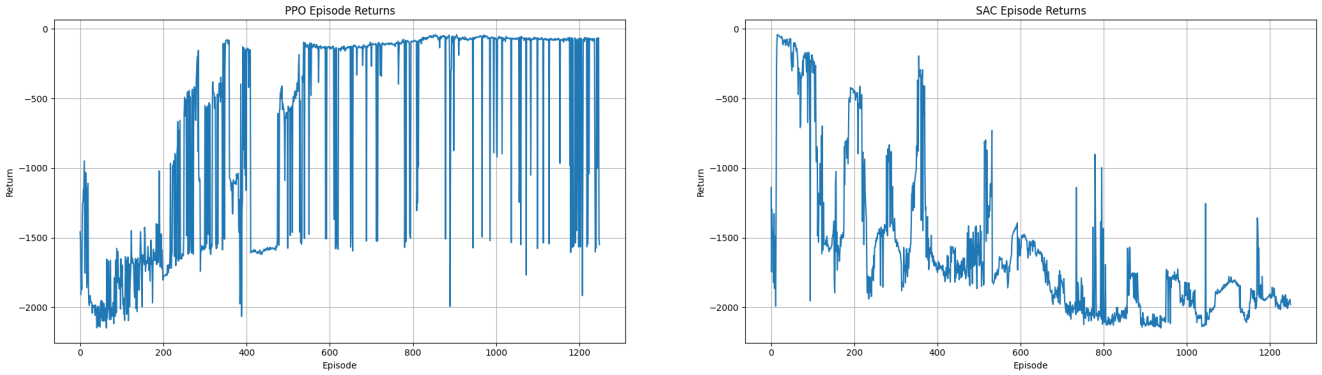


(a) Mean rewards PPO.

(b) Mean rewards SAC.

Figura 7: Mean rewards durante el entrenamiento para ambos algoritmos.

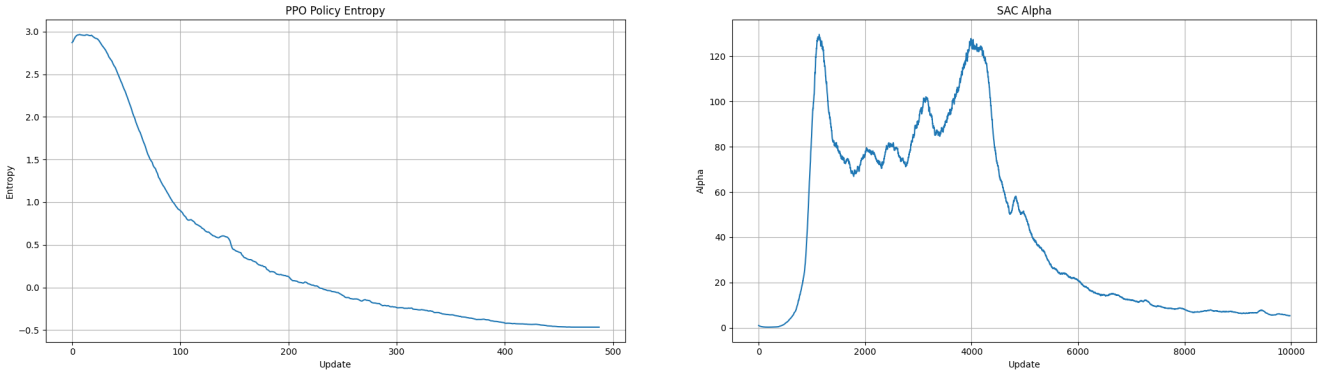
Se puede apreciar en estas figuras cómo PPO tiende a mejorar sus rewards acumulados, a diferencia de SAC. Respecto a la exploración, podemos ver cómo ambos tienen una política altamente explorativa al principio (SAC un poco más retardadamente), que luego va decreciendo hasta prácticamente ser determinista (política aprendida es mejor opción).



(a) Episodic Returns PPO.

(b) Episodic Returns SAC.

Figura 8: Episode returns durante el entrenamiento para ambos algoritmos.



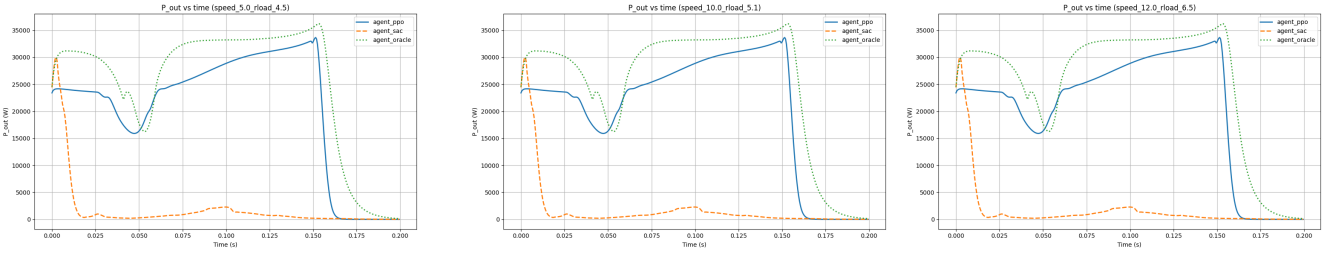
(a) Entropía de la política aprendida por PPO.

(b) Valor de alpha durante el entrenamiento de SAC.

Figura 9: Entropía de PPO y alpha de SAC durante el entrenamiento, ambos miden nivel de exploración de los algoritmos.

Para comparar el desempeño de estas políticas a un nivel más minuscioso, se hicieron rollouts de las políticas aprendidas para 3 escenarios distintos (combinaciones velocidad y R_{load}), graficando respecto a la posición:

$$(v, R_{load}) = \{(5, 4.5), (10, 5.1), (12, 6.5)\}$$

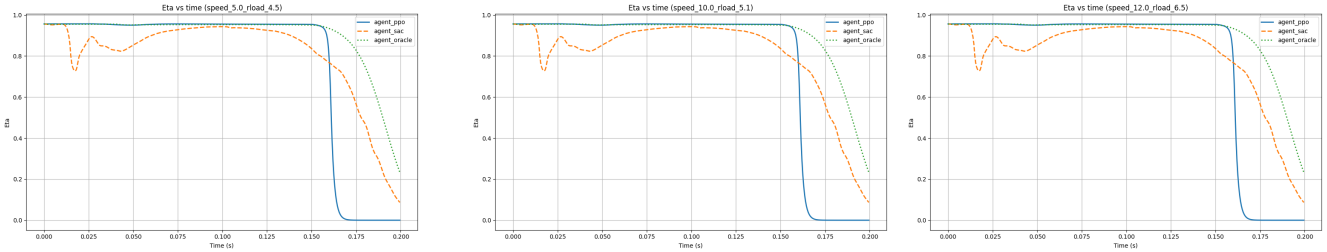


(a) $(v, R_{load}) = (5, 4.5)$

(b) $(v, R_{load}) = (10, 5.1)$

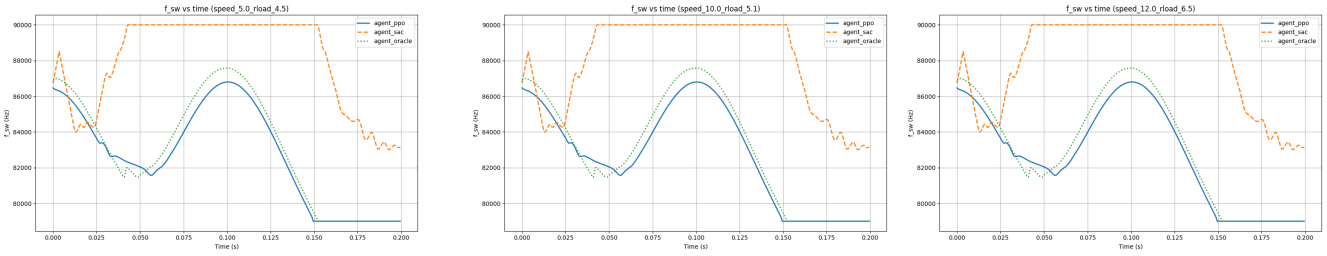
(c) $(v, R_{load}) = (12, 6.5)$

Figura 10: Comparación de potencia transmitida a la batería por el agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando FULL.



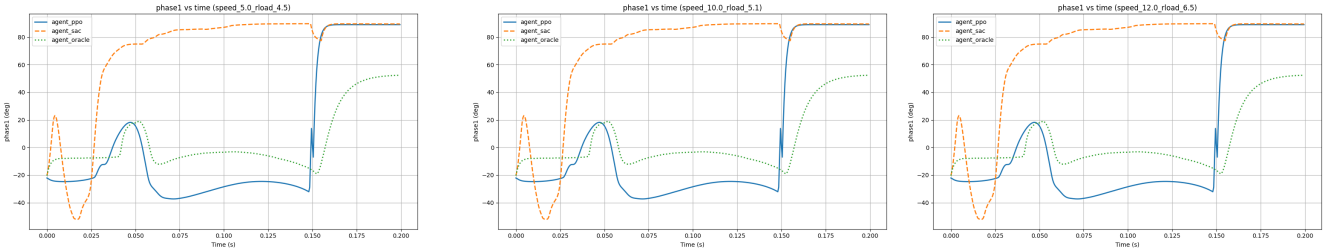
(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 11: Comparación de la eficiencia del sistema del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando FULL.



(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 12: Comparación de la frecuencia de conmutación del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando FULL.



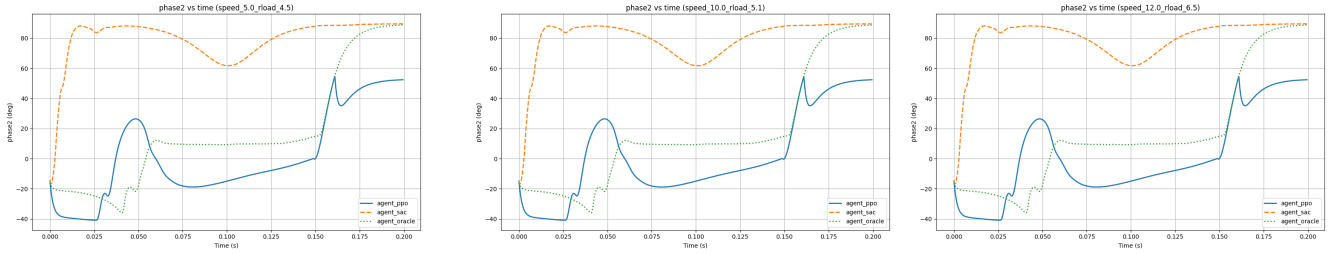
(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 13: Comparación de la primera fase del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando FULL.

Analizando todas las figuras de seguimiento del controlador oráculo, se puede apreciar claramente como PPO genera respuestas mucho mejores y más similares a las de SAC respecto al controlador oráculo. En general, ambas pasado suficiente tiempo son capaces de alcanzar al oráculo, pero la forma general y el tiempo de adaptación es mucho mejor en PPO. Pese a que SAC está estructuralmente mejor dotado para lidiar con espacios de acción continuos, PPO en este caso supera a la implementación de SAC, probablemente debido a sensibilidades con los hiperparámetros de entrenamiento de SAC.

(d)

En el caso de system knowledge = `no_comm`, se replanteó la función de rewards para ajustar por la falta de información acerca del secundario. La mejor forma de realizar esto es utilizar información de los



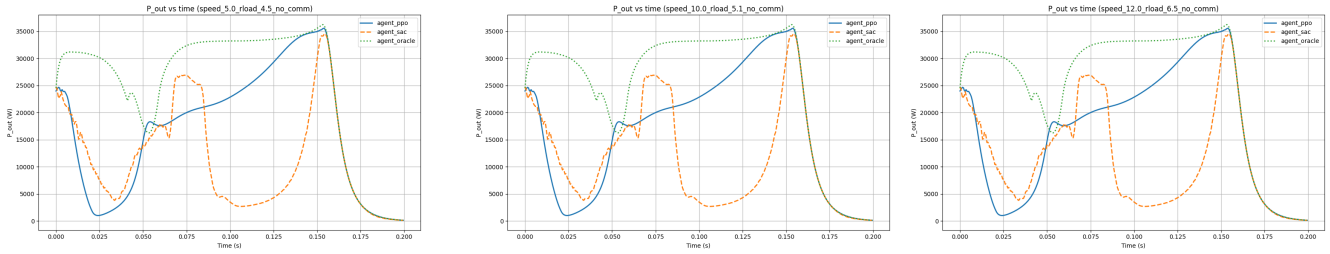
(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 14: Comparación de la segunda fase del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando FULL.

capacitores como fuente indirecta.

```python
    def compute_reward_no_comm(self):
        #solo vemos variables del primario (ni p_out no eta)
        power_term = self.pin / self.p.p_rated

        phase_penalty = phase_band_penalty(self.phase_active_mag_deg)

        capacitor_penalty = (
            max(0.0, self.vc1 - 6500.0) / 6500.0
            + max(0.0, self.vc2 - 6500.0) / 6500.0
        )

        reward = (
            1.0 * power_term
            - 0.25 * phase_penalty
            - 0.05 * capacitor_penalty
        )

        return float(reward)
```

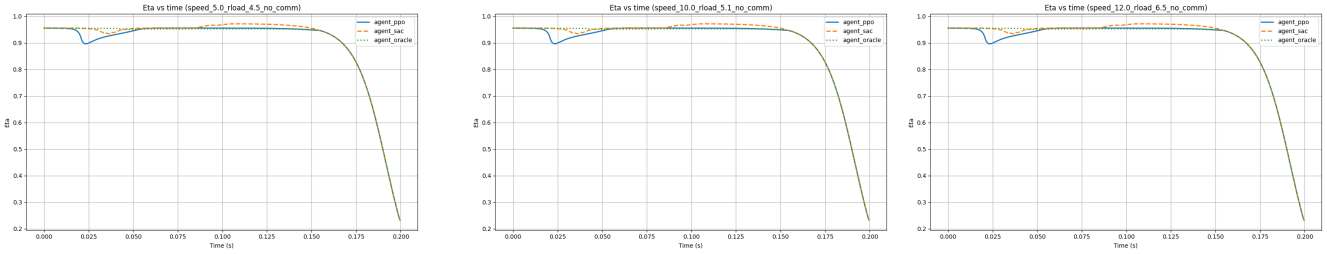
La información de potencia de salida se mide en el circuito primario (no directamente la percibida por el auto), la fase e información de los capacitores sirve para inferir que tan cerca de la resonancia se está operando (maximizando potencia y eficiencia). Así, se obtuvieron los siguientes resultados:



(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

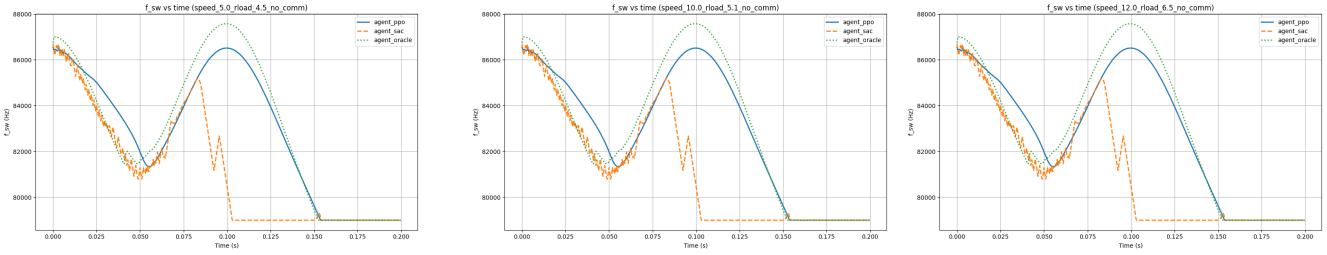
Figura 15: Comparación de potencia transmitida a la batería por el agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando no_comm.

Al analizar los resultados y comparar con el caso FULL, podemos ver que nuevamente PPO supera el rendimiento de SAC, acercándose significativamente a la curva del oráculo. La función de rewards diseñada es completamente adecuada, puesto que el rendimiento de los controladores es similar si no mejor que para



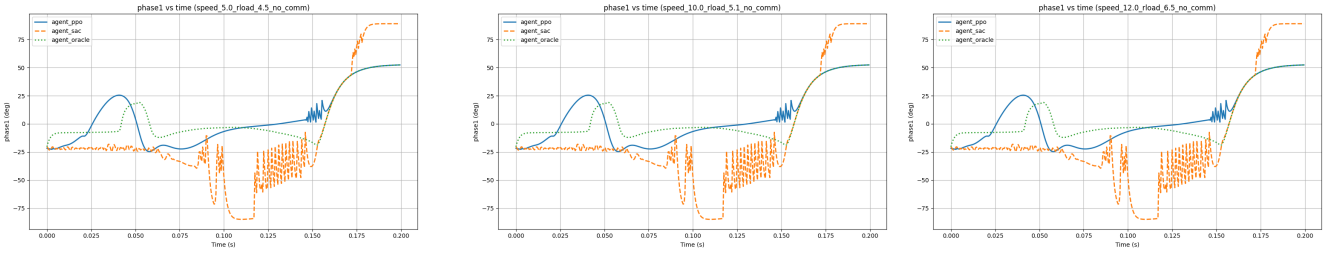
(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 16: Comparación de la eficiencia del sistema del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando no_comm.



(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 17: Comparación de la frecuencia de conmutación del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando no_comm.

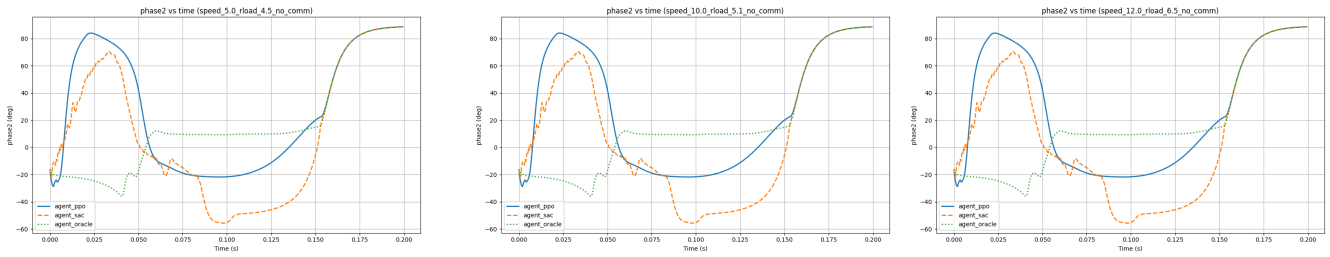


(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 18: Comparación de la primera fase del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando no_comm.

el caso FULL, demostrando que las variables y relaciones usadas son una buena representación para que el controlador pueda inferir la potencia transferida al auto. Nuevamente cabe recalcar que el rendimiento no tan bueno de SAC probablemente se debe a temas de hiperparámetros y no necesariamente a que el algoritmo sea mejor o peor, de hecho es probable que con buena calibración sea superior a PPO para espacios de acciones continuos.

Los códigos se encuentran en anexos (`p2_main.py` corre ambos entrenamientos, `sac.py` - entregado, `continous_action_ppo.py` adaptado de CleanRL[1]).



(a) $(v, R_{load}) = (5, 4.5)$ (b) $(v, R_{load}) = (10, 5.1)$ (c) $(v, R_{load}) = (12, 6.5)$

Figura 19: Comparación de la segunda fase del agente oráculo, entrenado por SAC, y entrenado por PPO para los distintos casos usando no_comm.

Pregunta 3

(a)

La función de rewards debe tomar en cuenta los objetivos del enunciado, además de incoporar el hecho de sólo se tiene acceso parcial al estado del sistema. Los principales objetivos deben ser:

- Mejorar el performance
- Evitar el sobreentrenamiento
- Promover la suavidad de entrenamiento entre días (no erráticos)
- Limitar los workouts intensos ($\omega_t < 2.5$)
- Promover la variedad de los ejercicios elegidos

El estado medible que el agente puede observar es:

$$s_t = (day/365,\ \texttt{restingHR},\ SpO_2,\ \texttt{benchmarkScore},\ \omega_{t-1}/\omega_{max})$$

Como tenemos acceso a la variable `true_performance` durante el entrenamiento, podemos dar reward directamente a esta variable. La función de rewards diseñada es entonces:

$$\begin{aligned} r_t =& ((P_t - P_{t-1}) \wedge (B_t - B_{t-1})) - \text{máx}(0, H_t - 5) - 0.1|\omega_t - \omega_{t-1}| \\ & - 0.2 \cdot ((HR_t - HR_{base})^+ + (SPO2_{base} - SPO2_t)^+) \\ & + 2D_t - 4 \cdot repeat - 3 \cdot type\_concentration - 6 \cdot consecutive\_repeat \end{aligned}$$

donde:

- En el primer término $(P_t - P_{t-1})$ premia la mejora de performance (true) respecto al día anterior y $(B_t - B_{t-1})$ premia la mejora en el benchmark test (B_t tiene ZOH hasta el próximo test) $\wedge$ indica si estamos usando el caso omnisciente o el baseline (con mediciones indirectas).
- El segundo término se compone de $H_t = \sum_{i=t-6}^{t} \mathbf{1}(\omega_i > 2.5)$, que es un contador del número de actividades de alta intensidad en la última semana. Si se alcanzan más de 5 de estas actividades, se activa un castigo para evitar este tipo de estrategias.
- El tercer término $|\omega_t - \omega_{t-1}|$ promueve suavidad entre entrenamientos (que el del día anterior sea similar al siguiente).

- En el penúltimo término $(HR_t - HR_{base})^+$ castiga siempre que haya una diferencia positiva entre el heart rate actual y el basal, inversamente que como $(SPO2_{base} - SPO2_t)^+$ lo hace con la saturación de oxígeno. Esto debido a que una menor saturación de O2 y un ritmo cardíaco en reposo mayor son indicios medibles que tenemos de fatiga corporal.
- Finalmente con $D_t =$ `# de types distintos en los últimos 7 días de entrenamiento` $\in \{0, 3\}$, promueve la diversidad en los entrenamientos escogidos. A esta variable, que por si sola debiese empujar hacia mayor variablidad, se le añadieron otras para penalizar mal comportamiento (el agente tendía a encontrar un workout favorito y explotarlo durante todo el año). Estos son *repeat* (penalización por repetir workouts durante la semana), *type_concetration* que penaliza la no-diversidad, y *consecutive_repeat* que castiga más por hacer el mismo workout en días consecutivos.

El flujo del controlador bajo esta función de rewards sería entonces:

1. Observación de entorno s_t
2. El controlador elije los parámetros ω_t y `type` del workout más deseables según la política aprendida.
3. A partir de esto, selecciona un workout específico de `rl_workouts_500.json` que cumpla con las especificaciones usando `workout_action()`.
4. Se aplica la acción, observa y actualiza la política basándose en la nueva medición del reward. La política se actualiza de la manera tradicional DDPG: el crítico estima la función de valor acción Q_θ sobre la cual evalúa el desempeño de la acción tomada por el actor, luego, sobre esta función estimada se calculan los gradientes que determinan hacia donde mover la política del actor. Q_θ se ajusta constantemente al igual que la política minimizando sus respectivas pérdidas.
5. Volver a 1.

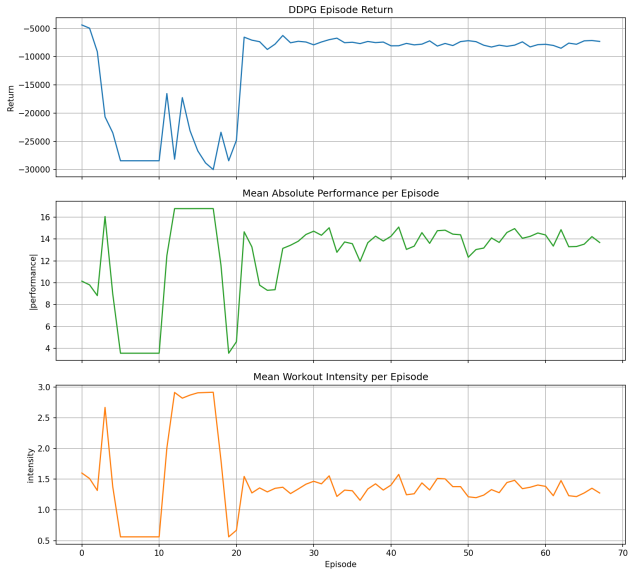
(b) / (c)

Se implementó DDPG utilizando un repositorio[2] que originialmente lo empleaba para un env de gymnasium. Se adaptó para que interactuara con los rewards y environment del problema. Los scripts `ddpg.py, ddpg_train.py, y ddpg_test.py` son los tomados del repositorio, y sólo se debe correr `p3.py` para que se entrenen y guarden los resultados pedidos. Se corren 6 entrenamientos y evaluaciones: dos iteraciones (omni y no omni) por cada uno de los 3 individuos (low, medium, high). Esto es con o sin omnisciencia (acceso a P_t), y los individuos son todos los parámetros en low, medium, o high dependiendo del caso. Los resultados del entrenamiento y rollout para cada caso se presentan a continuación (Figura 20, Figura 21, Figura 22).

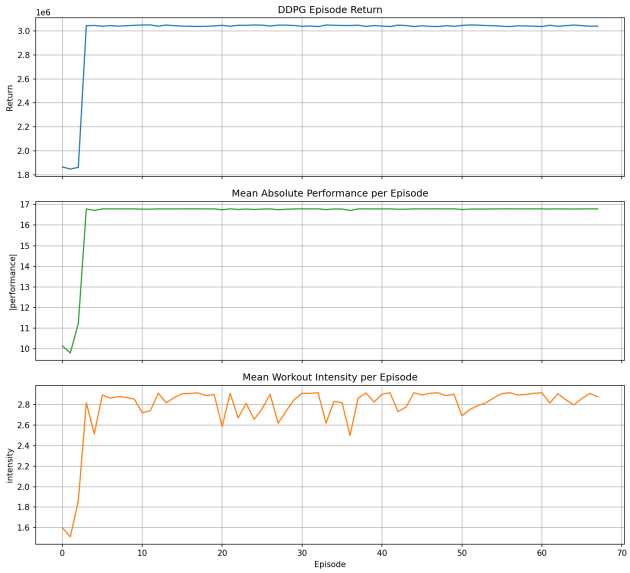
Analizando caso a caso, se puede apreciar cómo la omnisciencia no es realmente importante para la función de rewards diseñada. Puesto que el benchmark score es una muy buena representación del performance real. La función de rewards que sólo maximiza el performance (sin los otros indicadores) si logra efectivamente mejorar el score de performance de todos los individuos, teniendo mejores resultados en todos los casos. Sin embargo, al analizar las políticas que emplea cada método (omni vs no omni), se puede apreciar cada componente de la función de rewards generada provoca ciertos comportamientos.

Debido a que la versión de los rewards con mediciones indirectas era difícil de calibrar y que obtuviera resultados deseables, es más bien una función punitiva que una de recompensa. Es decir, se castigan muchos comportamientos no deseados más que premiar los retornos medibles ruidosamente. Esto provoca que los rewards sean muy negativos, a pesar de que el comportamiento efectivamente lleva a performances buenos y competitivos con la omnisciencia. En el caso omnisciente se puede ver cómo el algoritmo encuentra una intensidad óptima y la usa casi siempre para llegar a un performance alto. Esto no varía ni alterna entre

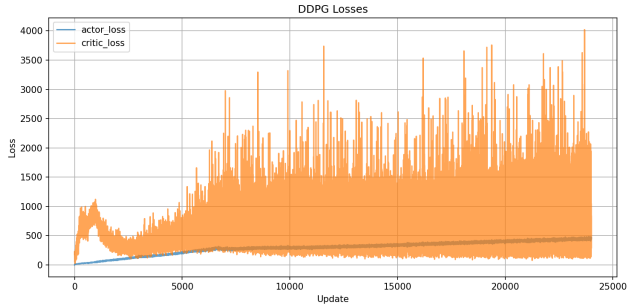
ejercicios, pero sí aumenta el performance. Como el caso no omnisciente tiene muchos beneficios por intentar de diversificar los ejercicios, se puede ver en el rollout de todos los casos que varía de tipo mucho más que el caso omnisciente (pero sí encuentra una intensidad óptima sobre la cual varía poco). El único caso extraño es el medium, donde la función de rewards (diseñada con $B_t - B_{t-1}$ como reward) da lugar a que oscilaciones aumenten el retorno (comportamiento no tan deseable). De todas maneras, en este caso el caso no omnisciente sigue siendo competitivo alcanzando alrededor del 75 % del performance del caso omnisciente.



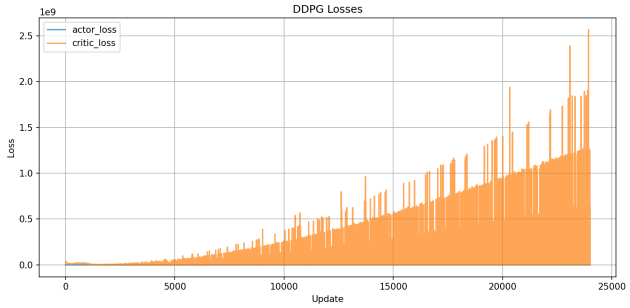
(a) Episode Metrics - Baseline



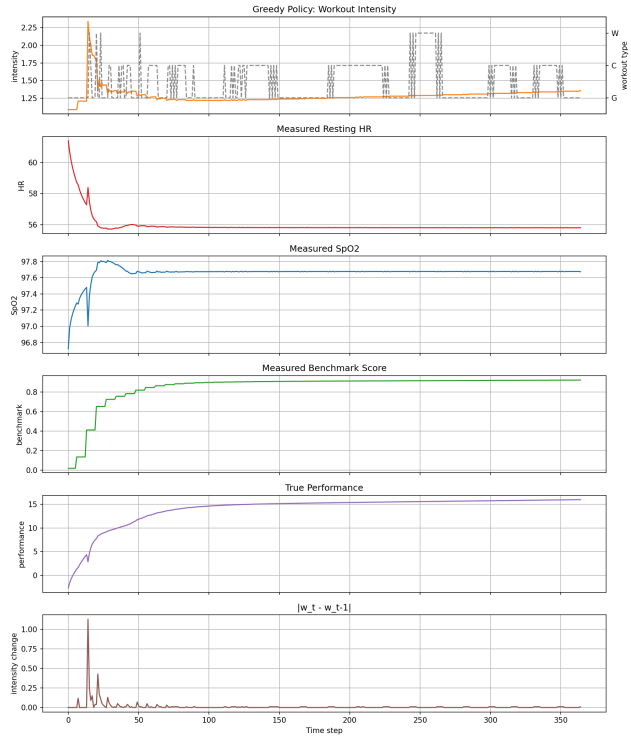
(b) Episode Metrics - Omni



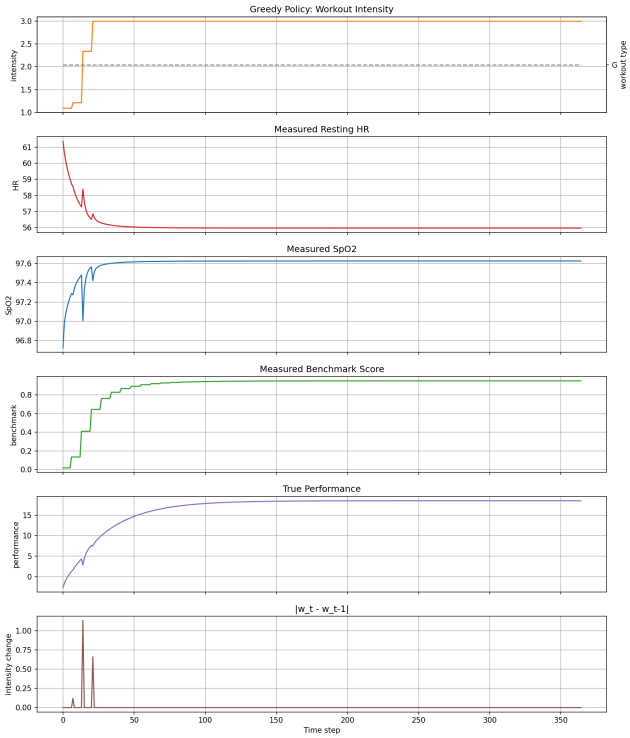
(c) Losses - Baseline



(d) Losses - Omni

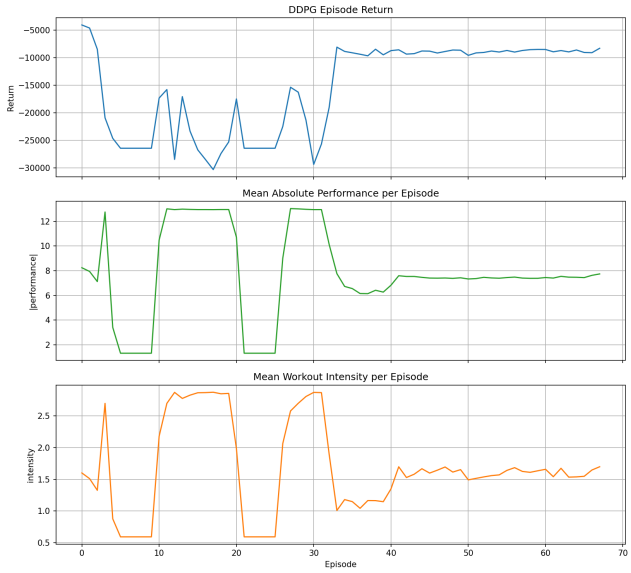


(e) Policy Rollout - Baseline

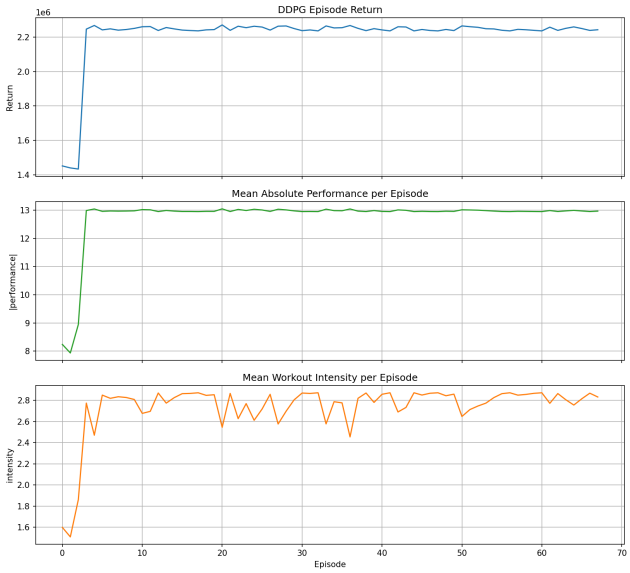


(f) Policy Rollout - Omni

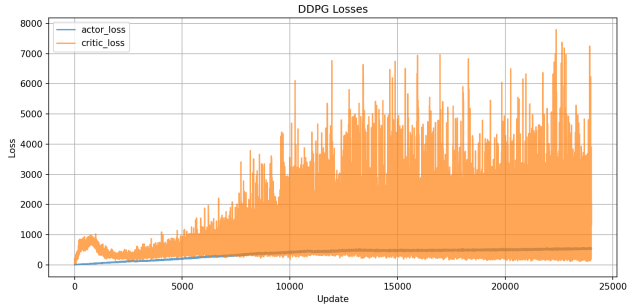
Figura 20: Resultados para el individuo HIGH.



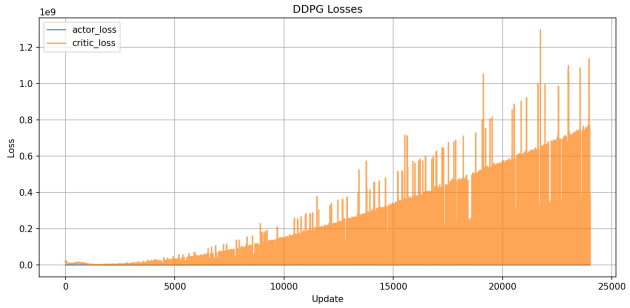
(a) Episode Metrics - Baseline



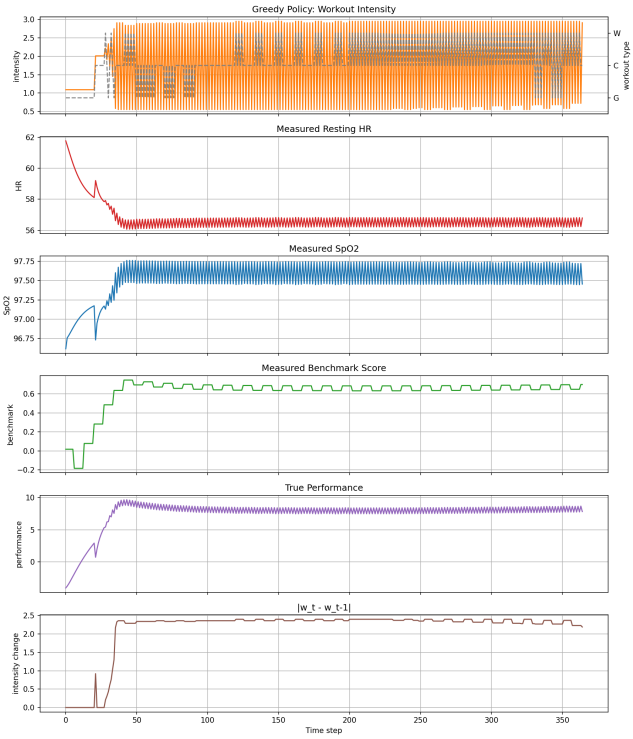
(b) Episode Metrics - Omni



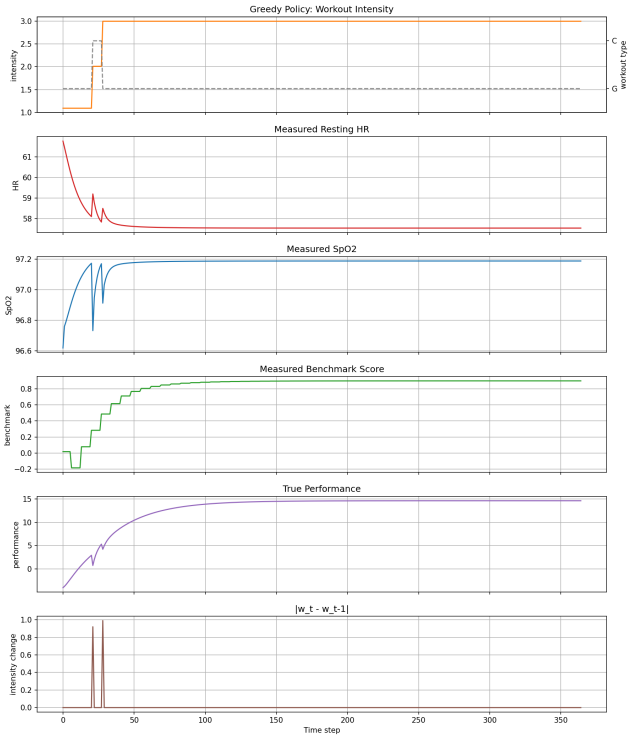
(c) Losses - Baseline



(d) Losses - Omni

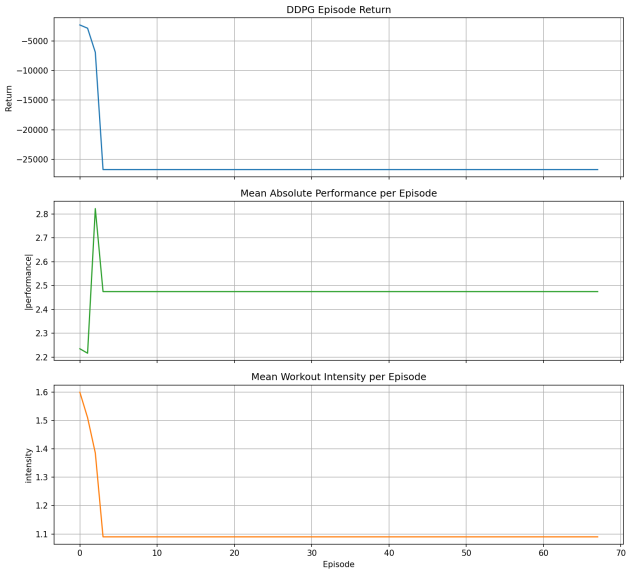


(e) Policy Rollout - Baseline

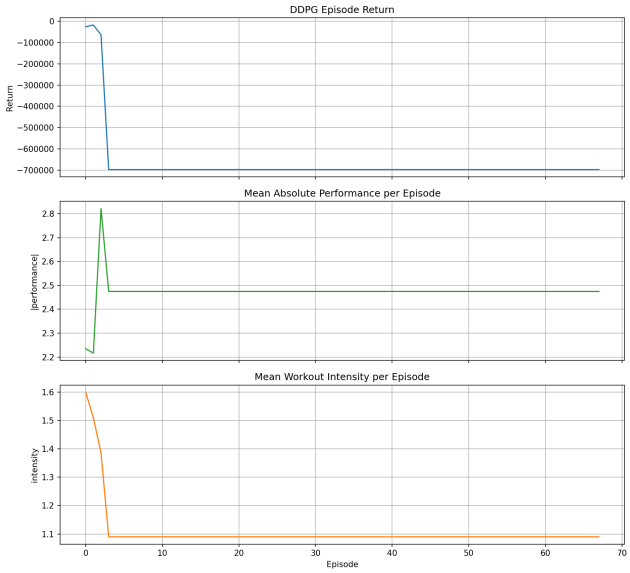


(f) Policy Rollout - Omni

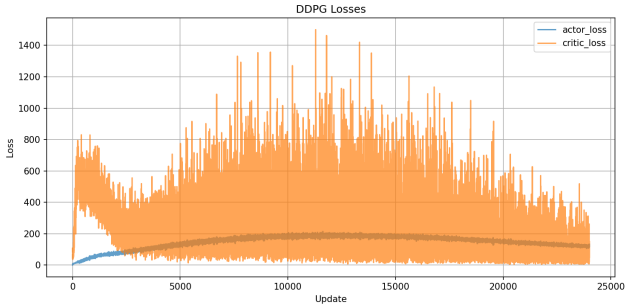
Figura 21: Resultados para el individuo MEDIUM.



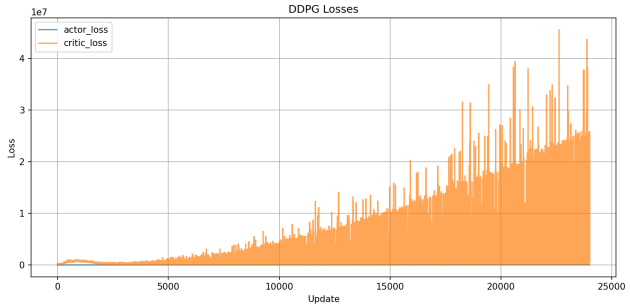
(a) Episode Metrics - Baseline



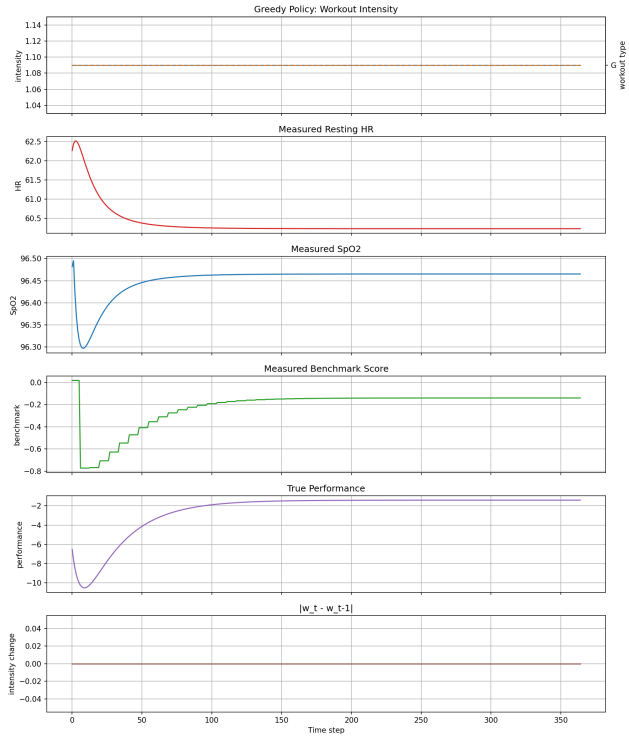
(b) Episode Metrics - Omni



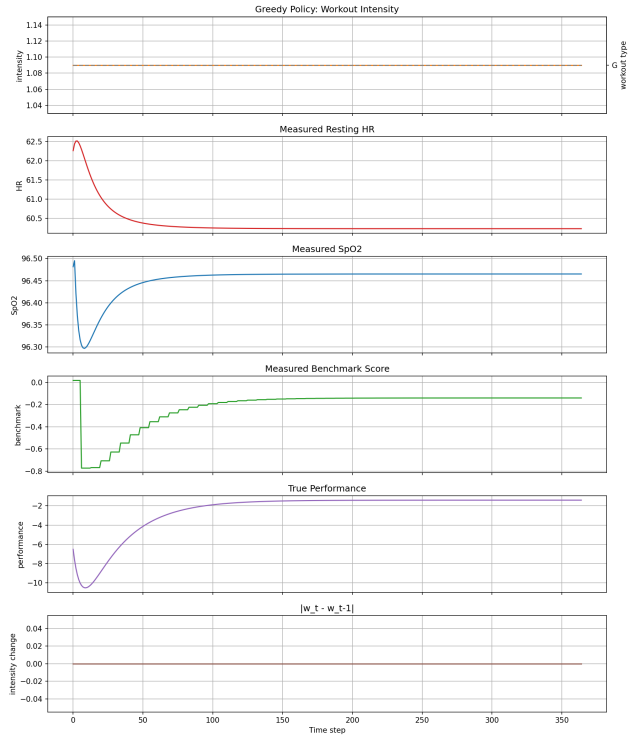
(c) Losses - Baseline



(d) Losses - Omni



(e) Policy Rollout - Baseline



(f) Policy Rollout - Omni

Figura 22: Resultados para el individuo LOW.

Anexos

Anexos Pregunta 2.

p2_main.py

```python
import os

from dwpt_rl import DWPTContinuousShareEnv, DWPTParams, oracle_controller
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from ppo_continuous_action import *
from sac import SACConfig, train as train_sac
import torch

print("\n" + "="*80)
print("Escenario a correr:")
print("  [1] full")
print("  [2] no_comm")
print("="*80)
user_choice = input("1 o 2?").strip()

if user_choice == "2":
    system_knowledge = 'no_comm'
    suffix = "_no_comm"
    print(f"Running NO_COMM\n")
else:
    system_knowledge = 'full'
    suffix = ""
    print(f"Running FULL\n")

params = DWPTParams()
env = DWPTContinuousShareEnv(params=params, seed=0, system_knowledge=system_knowledge)
env.reset(variable=False)

base_dir = os.path.dirname(os.path.abspath(__file__))
path = os.path.join(base_dir, f"results{suffix}")
os.makedirs(path, exist_ok=True)

# ================================================================================
# k1 y k2 vs posici n
positions = np.linspace(0, env.p.coil_spacing_m * 2, 200)
k1_values = []
k2_values = []

for pos in positions:
    k1, k2 = env.coupling_profile(pos)
    k1_values.append(k1)
    k2_values.append(k2)

plt.figure(figsize=(10, 6))
plt.plot(positions, k1_values, 'b-', label='k1 (acoplamiento coil1-secondary)',
    linewidth=2)
plt.plot(positions, k2_values, 'r-', label='k2 (acoplamiento coil2-secondary)',
    linewidth=2)
```

```
plt.axvline(x=0, color='b', linestyle='--', alpha=0.5)
plt.axvline(x=env.p.coil_spacing_m, color='r', linestyle='--', alpha=0.5)
plt.xlabel('Posici n del veh culo (m)', fontsize=12)
plt.ylabel('Coeficiente de acoplamiento (k)', fontsize=12)
plt.title('Perfil de acoplamiento magn tico', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(path, 'coupling_profile.png'))
plt.close()

# ================================================================================
# P vs f para distintos valores de k1
frequencies = np.linspace(env.p.f_min, env.p.f_max, 100)
k1_test_values = [0.05, 0.10, 0.15, 0.20, 0.26]

plt.figure(figsize=(12, 6))
for k1_test in k1_test_values:
    p_out_values = []
    for f in frequencies:
        sol = env.solve_network(f_sw=f, k1=k1_test, k2=0.0, share=0.0)
        p_out_values.append(sol["P_out_target"])
    plt.plot(frequencies / 1e3, p_out_values, marker='o', markersize=3, label=f'k1={
        k1_test:.2f}', linewidth=2)

plt.xlabel('Frecuencia de conmutaci n (kHz)', fontsize=12)
plt.ylabel('Potencia de salida (W)', fontsize=12)
plt.title('Potencia transferida vs Frecuencia (share=0)', fontsize=14, fontweight='bold'
    )
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(path, 'power_vs_freq.png'))
plt.close()

# ================================================================================
# eta vs f para distintos valores de k1
plt.figure(figsize=(12, 6))
for k1_test in k1_test_values:
    eta_values = []
    for f in frequencies:
        sol = env.solve_network(f_sw=f, k1=k1_test, k2=0.0, share=0.0)
        eta_values.append(sol["eta_target"])
    plt.plot(frequencies / 1e3, eta_values, marker='o', markersize=3, label=f'k1={
        k1_test:.2f}', linewidth=2)

plt.xlabel('Frecuencia de conmutaci n (kHz)', fontsize=12)
plt.ylabel('Eficiencia (  )', fontsize=12)
plt.title('Eficiencia vs Frecuencia (share=0)', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.ylim([0, 1])
plt.tight_layout()
plt.savefig(os.path.join(path, 'efficiency_vs_freq.png'))
plt.close()

# ================================================================================
# P vs phi barriendo frecuencia vs k1
plt.figure(figsize=(12, 6))
```

```python
for k1_test in k1_test_values:
    p_out_phase = []
    phase_values = []
    for f in frequencies:
        sol = env.solve_network(f_sw=f, k1=k1_test, k2=0.0, share=0.0)
        p_out_phase.append(sol["P_out_target"])
        phase_values.append(sol["phase_active_mag_deg"])
    plt.plot(phase_values, p_out_phase, marker='o', markersize=3, label=f'k1={k1_test:.2f}', linewidth=2)

plt.xlabel('Desfase de corriente (grados)', fontsize=12)
plt.ylabel('Potencia de salida (W)', fontsize=12)
plt.title('Potencia vs Desfase (variando frecuencia, share=0)', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(path, 'power_vs_phase.png'))
plt.close()

# ================================================================================
# P vs share vs pares (k1, k2)
shares = np.linspace(0.0, 1.0, 50)
k_pairs = [(0.05, 0.20), (0.15, 0.15), (0.20, 0.05), (0.26, 0.26)]

plt.figure(figsize=(12, 6))
for k1_test, k2_test in k_pairs:
    p_out_share = []
    for share in shares:
        sol = env.solve_network(f_sw=86.5e3, k1=k1_test, k2=k2_test, share=share)
        p_out_share.append(sol["P_out_target"])
    plt.plot(shares, p_out_share, marker='o', markersize=4, label=f'k1={k1_test:.2f}, k2={k2_test:.2f}', linewidth=2)

plt.xlabel('Distribuci n de potencia (share)', fontsize=12)
plt.ylabel('Potencia de salida (W)', fontsize=12)
plt.title('Potencia vs Share (frecuencia fija en 86.5 kHz)', fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(path, 'power_vs_share.png'))
plt.close()

##############
# Entrenamiento PPO + SAC

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("==============================")
print("          PPO                 ")
print("==============================")

args = Args()
ppo_agent, ppo_history = train_ppo(args, system_knowledge=system_knowledge)

print("==============================")
print("          SAC                 ")
print("==============================")
```

```
162
163 sac_cfg = SACConfig(seed=args.seed, total_steps=args.total_timesteps)
164 sac_env = make_env(system_knowledge=system_knowledge)
165 sac_history, sac_agent = train_sac(sac_env, sac_cfg)
166
167 # ============================
168 # Comparar PPO vs SAC vs or culo
169 # ============================
170 combos = [
171     (5.0, 4.5),
172     (10.0, 5.1),
173     (12.0, 6.5),
174 ]
175
176
177 def run_controller_on_condition(controller_kind, controller_obj, env, device):
178     obs, _ = env.reset(seed=0)
179     obs_dict = env.unwrapped._get_state('dict')
180     done = False
181     times = []
182     pouts = []
183     etas = []
184     fs = []
185     shares = []
186     phase1s = []
187     phase2s = []
188     t = 0.0
189
190     while not done:
191         if controller_kind == 'oracle':
192             action = oracle_controller(obs_dict, env.unwrapped)
193         elif controller_kind == 'ppo':
194             obs_arr = np.asarray(obs, dtype=np.float32)
195             obs_tensor = torch.as_tensor(obs_arr, device=device)
196             with torch.no_grad():
197                 action = controller_obj.get_deterministic_action(obs_tensor)
198             action = action.cpu().numpy().squeeze()
199         elif controller_kind == 'sac':
200             obs_arr = np.asarray(obs, dtype=np.float32)
201             action = controller_obj.act(obs_arr, deterministic=True)
202         else:
203             raise ValueError(f"Unknown controller kind: {controller_kind}")
204
205         obs, reward, terminated, truncated, info = env.step(action)
206         done = bool(terminated or truncated)
207
208         times.append(t)
209         pouts.append(env.unwrapped.pout)
210         etas.append(env.unwrapped.eta)
211         fs.append(env.unwrapped.f_sw)
212         shares.append(env.unwrapped.share)
213         phase1s.append(env.unwrapped.phase1_deg)
214         phase2s.append(env.unwrapped.phase2_deg)
215
216         t += env.unwrapped.p.dt
217         obs_dict = env.unwrapped._get_state('dict')
218
219     return {
220         'time': np.array(times),
```

```
        'pout': np.array(pouts),
        'eta': np.array(etas),
        'f_sw': np.array(fs),
        'share': np.array(shares),
        'phase1_deg': np.array(phase1s),
        'phase2_deg': np.array(phase2s),
    }


def plot_three_way_comparison(results_by_label, metric_key, ylabel, title, output_file):
    styles = {
        'agent_ppo': {'label': 'agent_ppo', 'linestyle': '-'},
        'agent_sac': {'label': 'agent_sac', 'linestyle': '--'},
        'agent_oracle': {'label': 'agent_oracle', 'linestyle': ':'},
    }
    plt.figure(figsize=(10, 6))
    for label, result in results_by_label.items():
        plt.plot(
            result['time'],
            result[metric_key],
            label=styles[label]['label'],
            linestyle=styles[label]['linestyle'],
            linewidth=2,
        )
    plt.xlabel('Time␣(s)')
    plt.ylabel(ylabel)
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(output_file)
    plt.close()


for speed, r_load in combos:
    params_tmp = DWPTParams()
    params_tmp.speed_mps = speed
    params_tmp.r_load_dc = r_load

    eval_env_ppo = make_env(system_knowledge=system_knowledge)
    eval_env_sac = make_env(system_knowledge=system_knowledge)
    eval_env_oracle = DWPTContinuousShareEnv(params=params_tmp, seed=0, system_knowledge
        =system_knowledge)

    # force same physical condition on all environments
    eval_env_ppo.unwrapped.p.speed_mps = speed
    eval_env_ppo.unwrapped.p.r_load_dc = r_load
    eval_env_sac.unwrapped.p.speed_mps = speed
    eval_env_sac.unwrapped.p.r_load_dc = r_load

    res_ppo = run_controller_on_condition('ppo', ppo_agent, eval_env_ppo, device)
    res_sac = run_controller_on_condition('sac', sac_agent, eval_env_sac, device)
    res_oracle = run_controller_on_condition('oracle', None, eval_env_oracle, device)

    label_suffix = f"speed_{speed}_rload_{r_load}{suffix}"
    results_by_label = {
        'agent_ppo': res_ppo,
        'agent_sac': res_sac,
        'agent_oracle': res_oracle,
```

```
    }

    plot_three_way_comparison(
        results_by_label,
        'pout',
        'P_out (W)',
        f'P_out vs time ({label_suffix})',
        os.path.join(path, f'compare_pout_{label_suffix}.png'),
    )
    plot_three_way_comparison(
        results_by_label,
        'eta',
        'Eta',
        f'Eta vs time ({label_suffix})',
        os.path.join(path, f'compare_eta_{label_suffix}.png'),
    )
    plot_three_way_comparison(
        results_by_label,
        'f_sw',
        'f_sw (Hz)',
        f'f_sw vs time ({label_suffix})',
        os.path.join(path, f'compare_fsw_{label_suffix}.png'),
    )
    plot_three_way_comparison(
        results_by_label,
        'share',
        'share',
        f'share vs time ({label_suffix})',
        os.path.join(path, f'compare_share_{label_suffix}.png'),
    )
    plot_three_way_comparison(
        results_by_label,
        'phase1_deg',
        'phase1 (deg)',
        f'phase1 vs time ({label_suffix})',
        os.path.join(path, f'compare_phase1_{label_suffix}.png'),
    )
    plot_three_way_comparison(
        results_by_label,
        'phase2_deg',
        'phase2 (deg)',
        f'phase2 vs time ({label_suffix})',
        os.path.join(path, f'compare_phase2_{label_suffix}.png'),
    )

    eval_env_ppo.close()
    eval_env_sac.close()
    eval_env_oracle.close()


################################################################################
# TRAINING PLOTS
################################################################################

plt.figure(figsize=(10, 6))
plt.plot(ppo_history["episode_returns"])
plt.xlabel("Episode")
plt.ylabel("Return")
plt.title("PPO Episode Returns")
```

```
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(path, "ppo_episode_returns.png"))
plt.close()

plt.figure(figsize=(10, 6))
plt.plot(ppo_history["mean_rewards"])
plt.xlabel("Iteration")
plt.ylabel("Mean Rollout Reward")
plt.title("PPO Mean Rollout Reward")
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(path, "ppo_mean_rewards.png"))
plt.close()

plt.figure(figsize=(10, 6))
plt.plot(ppo_history["policy_losses"])
plt.xlabel("Update")
plt.ylabel("Policy Loss")
plt.title("PPO Policy Loss")
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(path, "ppo_policy_loss.png"))
plt.close()

plt.figure(figsize=(10, 6))
plt.plot(ppo_history["value_losses"])
plt.xlabel("Update")
plt.ylabel("Value Loss")
plt.title("PPO Value Loss")
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(path, "ppo_value_loss.png"))
plt.close()

plt.figure(figsize=(10, 6))
plt.plot(ppo_history["entropy_losses"])
plt.xlabel("Update")
plt.ylabel("Entropy")
plt.title("PPO Policy Entropy")
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(path, "ppo_entropy.png"))
plt.close()


def save_line_plot(y_values, title, xlabel, ylabel, output_name):
    plt.figure(figsize=(10, 6))
    plt.plot(y_values)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(os.path.join(path, output_name))
    plt.close()


save_line_plot(
```

```
    [ret for _, ret in sac_history["episode_returns"]],
    "SAC Episode Returns",
    "Episode",
    "Return",
    "sac_episode_returns.png",
)
save_line_plot(
    sac_history["mean_rewards"],
    "SAC Mean Rewards",
    "Episode",
    "Mean Reward",
    "sac_mean_rewards.png",
)
save_line_plot(
    sac_history["q_loss"],
    "SAC Q Loss",
    "Update",
    "Q Loss",
    "sac_q_loss.png",
)
save_line_plot(
    sac_history["pi_loss"],
    "SAC Policy Loss",
    "Update",
    "Policy Loss",
    "sac_pi_loss.png",
)
save_line_plot(
    sac_history["alpha"],
    "SAC Alpha",
    "Update",
    "Alpha",
    "sac_alpha.png",
)
save_line_plot(
    sac_history["alpha_loss"],
    "SAC Alpha Loss",
    "Update",
    "Alpha Loss",
    "sac_alpha_loss.png",
)
save_line_plot(
    sac_history["logp_pi"],
    "SAC Log Prob Policy",
    "Update",
    "Logp Pi",
    "sac_logp_pi.png",
)
save_line_plot(
    sac_history["q1_mean"],
    "SAC Q1 Mean",
    "Update",
    "Q1 Mean",
    "sac_q1_mean.png",
)
```

ppo.py

```python
# adaptado de cleanrl\ppo_continuous_action.py
# docs and experiment results can be found at https://docs.cleanrl.dev/rl-algorithms/ppo/#ppo_continuous_actionpy
import os
import random
import time
from dataclasses import dataclass
from dwpt_rl import DWPTContinuousShareEnv

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions.normal import Normal
from torch.utils.tensorboard import SummaryWriter


@dataclass
class Args:
    seed: int = 1
    cuda: bool = True

    total_timesteps: int = 500_000

    learning_rate: float = 3e-4

    num_steps: int = 1024

    gamma: float = 0.99
    gae_lambda: float = 0.95

    num_minibatches: int = 8
    update_epochs: int = 10

    clip_coef: float = 0.2
    ent_coef: float = 0.01
    vf_coef: float = 0.5

    max_grad_norm: float = 0.5

    anneal_lr: bool = True

    save_model: bool = True

    # runtime filled
    batch_size: int = 0
    minibatch_size: int = 0
    num_iterations: int = 0


class DWPTEnvAdapter(gym.Wrapper):
    """Adapter to make DWPTContinuousShareEnv compatible with gymnasium API."""
    def reset(self, seed=None, options=None):
        # Call parent reset ignoring options (DWPTContinuousShareEnv doesn't support it)
        obs, info = self.env.reset(seed=seed)
        # Convert obs to numpy array if dict
        if isinstance(obs, dict):
```

```
            obs = np.array(list(obs.values()), dtype=np.float32)
        else:
            obs = np.array(obs, dtype=np.float32)
        return obs, info

    def step(self, action):
        obs, reward, terminated, truncated, info = self.env.step(action)
        # Convert obs to numpy array if dict
        if isinstance(obs, dict):
            obs = np.array(list(obs.values()), dtype=np.float32)
        else:
            obs = np.array(obs, dtype=np.float32)
        return obs, float(reward), bool(terminated), bool(truncated), info


def make_env(system_knowledge='full'):
    env = DWPTContinuousShareEnv(system_knowledge=system_knowledge)
    env = DWPTEnvAdapter(env)
    env = gym.wrappers.RecordEpisodeStatistics(env)
    env = gym.wrappers.NormalizeObservation(env)
    return env


def layer_init(layer, std=np.sqrt(2), bias_const=0.0):
    torch.nn.init.orthogonal_(layer.weight, std)
    torch.nn.init.constant_(layer.bias, bias_const)
    return layer


class Agent(nn.Module):
    def __init__(self, obs_dim, action_dim):
        super().__init__()
        self.critic = nn.Sequential(
            layer_init(nn.Linear(obs_dim, 128)),
            nn.Tanh(),
            layer_init(nn.Linear(128, 128)),
            nn.Tanh(),
            layer_init(nn.Linear(128, 1), std=1.0),
        )
        self.actor_mean = nn.Sequential(
            layer_init(nn.Linear(obs_dim, 128)),
            nn.Tanh(),
            layer_init(nn.Linear(128, 128)),
            nn.Tanh(),
            layer_init(nn.Linear(128, action_dim), std=0.01),
        )
        self.actor_logstd = nn.Parameter(torch.zeros(1, action_dim))

    def get_value(self, x):
        if x.dim() == 1:
            x = x.unsqueeze(0)
        return self.critic(x)

    def get_action_and_value(self, x, action=None):
        if x.dim() == 1:
            x = x.unsqueeze(0)
        action_mean = self.actor_mean(x)
        action_logstd = self.actor_logstd.expand_as(action_mean)
        action_std = torch.exp(action_logstd)
```

```
        probs = Normal(action_mean, action_std)
        if action is None:
            action = probs.sample()
        logprob = probs.log_prob(action).sum(-1)
        entropy = probs.entropy().sum(-1)
        return action, logprob, entropy, self.critic(x)

    def get_deterministic_action(self, x):
        if x.dim() == 1:
            x = x.unsqueeze(0)
        action_mean = self.actor_mean(x)
        return torch.clamp(action_mean, -1.0, 1.0)


def train_ppo(args, system_knowledge='full'):
    # runtime batch sizes will be computed after env creation
    run_name = f"ppo_dwpt__seed{args.seed}__{int(time.time())}"
    writer = SummaryWriter(f"runs/{run_name}")
    writer.add_text(
        "hyperparameters",
        "|param|value|\n|-|-|\n%s" % ("\n".join([f"|{key}|{value}|" for key, value in
            vars(args).items()])),
    )

    # TRY NOT TO MODIFY: seeding
    random.seed(args.seed)
    np.random.seed(args.seed)
    torch.manual_seed(args.seed)
    torch.backends.cudnn.deterministic = True

    device = torch.device("cuda" if torch.cuda.is_available() and args.cuda else "cpu")

    # env setup (single env for easier debugging)
    env = make_env(system_knowledge=system_knowledge)
    assert isinstance(env.action_space, gym.spaces.Box), "only continuous action space
        is supported"

    obs_dim = int(np.prod(env.observation_space.shape))
    action_dim = int(np.prod(env.action_space.shape))

    # runtime batch sizes
    args.batch_size = int(args.num_steps)
    args.minibatch_size = int(args.batch_size // args.num_minibatches)
    args.num_iterations = int(args.total_timesteps // args.batch_size)

    agent = Agent(obs_dim, action_dim).to(device)
    optimizer = optim.Adam(agent.parameters(), lr=args.learning_rate, eps=1e-5)

    # ALGO Logic: Storage setup (single-env rollout)
    obs = torch.zeros((args.num_steps, obs_dim)).to(device)
    actions = torch.zeros((args.num_steps, action_dim)).to(device)
    logprobs = torch.zeros(args.num_steps).to(device)
    rewards = torch.zeros(args.num_steps).to(device)
    dones = torch.zeros(args.num_steps).to(device)
    values = torch.zeros(args.num_steps).to(device)

    # TRY NOT TO MODIFY: start the game
    global_step = 0
    start_time = time.time()
```

```
    next_obs, info = env.reset(seed=args.seed)
    next_obs = torch.Tensor(next_obs).to(device)
    next_done = 0

    # HISTORIES
    episode_returns = []
    value_losses = []
    policy_losses = []
    entropy_losses = []
    mean_rewards = []

    mean_eta = []
    mean_pout = []
    mean_phase = []

    # Episode counter for progress printing
    num_episodes = 0
    print(f"\n{'='*80}")
    print(f"Starting PPO training")
    print(f"  Total timesteps: {args.total_timesteps}")
    print(f"  Iterations: {args.num_iterations}")
    print(f"  Steps per rollout: {args.num_steps}")
    print(f"  Learning rate: {args.learning_rate}")
    print(f"  Device: {device}")
    print(f"{'='*80}\n")

    for iteration in range(1, args.num_iterations + 1):
        # Annealing the rate if instructed to do so.
        if args.anneal_lr:
            frac = 1.0 - (iteration - 1.0) / args.num_iterations
            lrnow = frac * args.learning_rate
            optimizer.param_groups[0]["lr"] = lrnow

        # rollout accumulators for physics metrics
        rollout_eta = []
        rollout_pout = []
        rollout_phase = []

        for step in range(0, args.num_steps):

            global_step += 1
            obs[step] = next_obs
            dones[step] = float(next_done)

            # ALGO LOGIC: action logic
            with torch.no_grad():
                action, logprob, _, value = agent.get_action_and_value(next_obs)
                # clamp actions to [-1,1] before env.step
                action = torch.clamp(action, -1.0, 1.0)
                values[step] = value.flatten()
            actions[step] = action.squeeze()
            logprobs[step] = logprob.squeeze()

            # execute the game and log data.
            action_np = action.cpu().numpy().squeeze()
            next_obs, reward, terminated, truncated, info = env.step(action_np)
            step_info = info

            next_done = bool(terminated or truncated)
```

```
            if next_done:
                next_obs, _ = env.reset()

            rewards[step] = torch.tensor(reward).to(device).view(-1)
            next_obs = torch.Tensor(next_obs).to(device)

            # collect physics metrics from info or env attributes
            if step_info and "eta" in step_info:
                rollout_eta.append(step_info.get("eta"))
            elif hasattr(env.unwrapped, "eta"):
                rollout_eta.append(getattr(env.unwrapped, "eta"))

            if step_info and "p_out" in step_info:
                rollout_pout.append(step_info.get("p_out"))
            elif hasattr(env.unwrapped, "pout"):
                rollout_pout.append(getattr(env.unwrapped, "pout"))

            if step_info and "phase_active_mag_deg" in step_info:
                rollout_phase.append(step_info.get("phase_active_mag_deg"))
            elif hasattr(env.unwrapped, "phase_active_mag_deg"):
                rollout_phase.append(getattr(env.unwrapped, "phase_active_mag_deg"))

            if step_info and "episode" in step_info:
                num_episodes += 1
                episode_returns.append(step_info["episode"]["r"])
                # Print every 10 episodes
                if num_episodes % 10 == 0:
                    avg_return = np.mean(episode_returns[-10:])
                    print(f"[Episode␣{num_episodes}]␣global_step={global_step},␣"
                          f"iteration={iteration}/{args.num_iterations},␣"
                          f"episodic_return={step_info['episode']['r']:.2f},␣"
                          f"avg_return_last_10={avg_return:.2f}")
                writer.add_scalar("charts/episodic_return", step_info["episode"]["r"],
                    global_step)
                writer.add_scalar("charts/episodic_length", step_info["episode"]["l"],
                    global_step)
                # physics logging
                if hasattr(env.unwrapped, "pout"):
                    writer.add_scalar("physics/pout", float(env.unwrapped.pout),
                        global_step)
                if hasattr(env.unwrapped, "eta"):
                    writer.add_scalar("physics/eta", float(env.unwrapped.eta),
                        global_step)
                if hasattr(env.unwrapped, "phase_active_mag_deg"):
                    writer.add_scalar("physics/phase", float(env.unwrapped.
                        phase_active_mag_deg), global_step)
                if hasattr(env.unwrapped, "share"):
                    writer.add_scalar("physics/share", float(env.unwrapped.share),
                        global_step)
                if hasattr(env.unwrapped, "f_sw"):
                    writer.add_scalar("physics/f_sw", float(env.unwrapped.f_sw),
                        global_step)

        # after rollout collection, record mean rollout reward and physics means
        try:
            mean_rewards.append(rewards.mean().item())
        except Exception:
            mean_rewards.append(float(torch.mean(rewards).item()))

```

```
        if len(rollout_eta) > 0:
            mean_eta.append(float(np.mean(rollout_eta)))
        else:
            mean_eta.append(float('nan'))
        if len(rollout_pout) > 0:
            mean_pout.append(float(np.mean(rollout_pout)))
        else:
            mean_pout.append(float('nan'))
        if len(rollout_phase) > 0:
            mean_phase.append(float(np.mean(rollout_phase)))
        else:
            mean_phase.append(float('nan'))

        # bootstrap value if not done
        with torch.no_grad():
            next_value = agent.get_value(next_obs).reshape(1, -1)
            advantages = torch.zeros_like(rewards).to(device)
            lastgaelam = 0
            for t in reversed(range(args.num_steps)):
                if t == args.num_steps - 1:
                    nextnonterminal = 1.0 - float(next_done)
                    nextvalues = next_value.squeeze()
                else:
                    nextnonterminal = 1.0 - dones[t + 1]
                    nextvalues = values[t + 1]
                delta = rewards[t] + args.gamma * nextvalues * nextnonterminal - values[
                    t]
                advantages[t] = lastgaelam = delta + args.gamma * args.gae_lambda *
                    nextnonterminal * lastgaelam
            returns = advantages + values

        # batch is already flat for single env rollouts
        b_obs = obs
        b_logprobs = logprobs
        b_actions = actions
        b_advantages = advantages
        b_returns = returns
        b_values = values

        # Optimizing the policy and value network
        b_inds = np.arange(args.batch_size)
        for epoch in range(args.update_epochs):
            np.random.shuffle(b_inds)
            for start in range(0, args.batch_size, args.minibatch_size):
                end = start + args.minibatch_size
                mb_inds = b_inds[start:end]

                _, newlogprob, entropy, newvalue = agent.get_action_and_value(b_obs[
                    mb_inds], b_actions[mb_inds])
                logratio = newlogprob - b_logprobs[mb_inds]
                ratio = logratio.exp()

                mb_advantages = b_advantages[mb_inds]
                if args.minibatch_size > 1:
                    mb_advantages = (mb_advantages - mb_advantages.mean()) / (
                        mb_advantages.std() + 1e-8)

                # Policy loss
                pg_loss1 = -mb_advantages * ratio
```

```
                pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - args.clip_coef, 1 + args.clip_coef)
                pg_loss = torch.max(pg_loss1, pg_loss2).mean()

                # Value loss
                newvalue = newvalue.view(-1)
                v_loss = 0.5 * ((newvalue - b_returns[mb_inds]) ** 2).mean()

                entropy_loss = entropy.mean()
                loss = pg_loss - args.ent_coef * entropy_loss + v_loss * args.vf_coef

                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(agent.parameters(), args.max_grad_norm)
                optimizer.step()

        # record PPO losses (use last computed batch losses)
        try:
            value_losses.append(v_loss.item())
        except Exception:
            value_losses.append(float('nan'))
        try:
            policy_losses.append(pg_loss.item())
        except Exception:
            policy_losses.append(float('nan'))
        try:
            entropy_losses.append(entropy_loss.item())
        except Exception:
            entropy_losses.append(float('nan'))

        # record rewards and a few key losses for plotting
        writer.add_scalar("charts/learning_rate", optimizer.param_groups[0]["lr"], global_step)
        writer.add_scalar("losses/value_loss", v_loss.item(), global_step)
        writer.add_scalar("losses/policy_loss", pg_loss.item(), global_step)
        writer.add_scalar("losses/entropy", entropy_loss.item(), global_step)

        # checkpoint every 10 iterations
        if iteration % 10 == 0:
            torch.save(agent.state_dict(), "ppo_dwpt.pt")
            elapsed = time.time() - start_time
            print(f"[Iteration {iteration}/{args.num_iterations}] "
                  f"Global step: {global_step}, "
                  f"Episodes: {num_episodes}, "
                  f"Elapsed: {elapsed:.1f}s, "
                  f"Last policy loss: {policy_losses[-1]:.4f}, "
                  f"Last value loss: {value_losses[-1]:.4f}")

    if args.save_model:
        model_path = f"runs/{run_name}/ppo_dwpt.cleanrl_model"
        torch.save(agent.state_dict(), model_path)
        print(f"model saved to {model_path}")

    env.close()
    writer.close()

    total_time = time.time() - start_time
    print(f"\n{'='*80}")
    print(f"Training completed!")
```

```
    print(f"␣␣Total␣episodes:␣{num_episodes}")
    print(f"␣␣Total␣timesteps:␣{global_step}")
    print(f"␣␣Total␣time:␣{total_time:.1f}s␣({total_time/60:.1f}m)")
    if len(episode_returns) > 0:
        print(f"␣␣Avg␣episode␣return:␣{np.mean(episode_returns):.2f}")
        print(f"␣␣Max␣episode␣return:␣{np.max(episode_returns):.2f}")
    else:
        print("␣␣Avg␣episode␣return:␣n/a␣(no␣completed␣episodes␣recorded)")
        print("␣␣Max␣episode␣return:␣n/a␣(no␣completed␣episodes␣recorded)")
    print(f"{'='*80}\n")

    history = {
        "episode_returns": episode_returns,
        "value_losses": value_losses,
        "policy_losses": policy_losses,
        "entropy_losses": entropy_losses,
        "mean_rewards": mean_rewards,
        "mean_eta": mean_eta,
        "mean_pout": mean_pout,
        "mean_phase": mean_phase,
    }

    return agent, history


def evaluate_policy(agent, env, device, num_episodes=5):
    agent.eval()

    trajectories = []

    for ep in range(num_episodes):
        obs, _ = env.reset()
        obs = torch.Tensor(obs).to(device)
        done = False

        ep_return = 0.0

        # per-episode trajectory storage
        freqs = []
        shares = []
        powers = []
        etas = []
        phases = []
        positions = []

        while not done:
            with torch.no_grad():
                action = agent.get_deterministic_action(obs)
            action_np = action.cpu().numpy().squeeze()
            obs, reward, terminated, truncated, info = env.step(action_np)
            done = bool(terminated or truncated)
            obs = torch.Tensor(obs).to(device)

            ep_return += float(reward)

            # collect trajectory info from env
            # safely read attributes from wrapped envs (NormalizeObservation,
                RecordEpisodeStatistics)
            def _env_attr(e, name):
```

```
                if hasattr(e, name):
                    return getattr(e, name)
                if hasattr(e, 'unwrapped') and hasattr(e.unwrapped, name):
                    return getattr(e.unwrapped, name)
                return float('nan')

            freqs.append(_env_attr(env, 'f_sw'))
            shares.append(_env_attr(env, 'share'))
            powers.append(_env_attr(env, 'pout'))
            etas.append(_env_attr(env, 'eta'))
            phases.append(_env_attr(env, 'phase_active_mag_deg'))
            positions.append(_env_attr(env, 'x_pos'))

        print(f"eval episode {ep} return: {ep_return}")
        trajectories.append({
            'return': ep_return,
            'freqs': freqs,
            'shares': shares,
            'powers': powers,
            'etas': etas,
            'phases': phases,
            'positions': positions,
        })

    return trajectories
```

sac.py

```
import random
import time
from dataclasses import dataclass
from typing import Dict, Tuple
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import gymnasium as gym

matplotlib.use("Agg")



def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)


def to_tensor(x, device):
    return torch.as_tensor(x, dtype=torch.float32, device=device)



def reward_function(obs):
```

```
    return 1


class ReplayBuffer:
    def __init__(self, obs_dim: int, act_dim: int, size: int, device: torch.device):
        self.device = device
        self.max_size = size
        self.ptr = 0
        self.len = 0

        self.obs      = np.zeros((size, obs_dim), dtype=np.float32)
        self.next_obs = np.zeros((size, obs_dim), dtype=np.float32)
        self.acts     = np.zeros((size, act_dim), dtype=np.float32)
        self.rews     = np.zeros((size, 1),       dtype=np.float32)
        self.done     = np.zeros((size, 1),       dtype=np.float32)

    def add(self, obs, act, rew, next_obs, done):
        self.obs[self.ptr]      = obs
        self.acts[self.ptr]     = act
        self.rews[self.ptr]     = rew
        self.next_obs[self.ptr] = next_obs
        self.done[self.ptr]     = done

        self.ptr = (self.ptr + 1) % self.max_size
        self.len = min(self.len + 1, self.max_size)

    def sample(self, batch_size: int):
        idx = np.random.randint(0, self.len, size=batch_size)
        return dict(
            obs      = to_tensor(self.obs[idx],      self.device),
            acts     = to_tensor(self.acts[idx],     self.device),
            rews     = to_tensor(self.rews[idx],     self.device),
            next_obs = to_tensor(self.next_obs[idx], self.device),
            done     = to_tensor(self.done[idx],     self.device),
        )


def mlp(sizes, activation=nn.ReLU, output_activation=nn.Identity):
    layers = []
    for i in range(len(sizes) - 1):
        act = activation if i < len(sizes) - 2 else output_activation
        layers += [nn.Linear(sizes[i], sizes[i + 1]), act()]
    return nn.Sequential(*layers)


class CriticQ(nn.Module):
    def __init__(self, obs_dim: int, act_dim: int, hidden_sizes=(128, 128, 128)):
        super().__init__()
        self.net = mlp([obs_dim + act_dim, *hidden_sizes, 1])

    def forward(self, obs, act):
        return self.net(torch.cat([obs, act], dim=-1))


LOG_STD_MIN = -20
LOG_STD_MAX = 2
LOG_STD_MIN = -5
LOG_STD_MAX = 2

```

```python
class SquashedGaussianActor(nn.Module):
    def __init__(self, obs_dim: int, act_dim: int, act_limit: float, hidden_sizes=(128, 128, 64)):
        super().__init__()
        self.act_limit = act_limit
        self.net = mlp([obs_dim, *hidden_sizes, 2 * act_dim])

    def forward(self, obs):
        mu_logstd = self.net(obs)
        mu, log_std = torch.chunk(mu_logstd, 2, dim=-1)
        log_std = torch.clamp(log_std, LOG_STD_MIN, LOG_STD_MAX)
        std = torch.exp(log_std)
        return mu, std

    def sample(self, obs):
        mu, std = self.forward(obs)
        dist = torch.distributions.Normal(mu, std)
        z = dist.rsample()
        a = torch.tanh(z)
        act = self.act_limit * a


        # log prob with tanh correction
        logp_z = dist.log_prob(z).sum(dim=-1, keepdim=True)
        correction = torch.log(1 - a.pow(2) + 1e-6).sum(dim=-1, keepdim=True)
        logp_a = logp_z - correction

        mu_act = self.act_limit * torch.tanh(mu)
        return act, logp_a, mu_act



@dataclass
class SACConfig:
    seed: int = 0

    total_steps: int  = 200_000
    start_steps: int  = 5_000
    update_after: int = 1_000
    update_every: int = 50
    batch_size: int   = 256
    replay_size: int  = 200_000

    gamma: float = 0.99
    tau:   float = 0.005

    actor_lr: float = 3e-4
    critic_lr: float = 3e-4
    alpha_lr: float  = 3e-4

    hidden_sizes: Tuple[int, int] = (256, 256)
    device: str = "cuda" if torch.cuda.is_available() else "cpu"

    eval_every:    int = 10_000
    eval_episodes: int = 5



class SACAgent:
```

```
    def __init__(self, obs_dim, act_dim, act_limit, cfg: SACConfig):
        self.cfg    = cfg
        self.device = torch.device(cfg.device)

        self.actor = SquashedGaussianActor(obs_dim, act_dim, act_limit, cfg.hidden_sizes
            ).to(self.device)
        self.q1    = CriticQ(obs_dim, act_dim, cfg.hidden_sizes).to(self.device)
        self.q2    = CriticQ(obs_dim, act_dim, cfg.hidden_sizes).to(self.device)

        self.q1_t = CriticQ(obs_dim, act_dim, cfg.hidden_sizes).to(self.device)
        self.q2_t = CriticQ(obs_dim, act_dim, cfg.hidden_sizes).to(self.device)
        self.q1_t.load_state_dict(self.q1.state_dict())
        self.q2_t.load_state_dict(self.q2.state_dict())

        self.actor_opt  = torch.optim.Adam(self.actor.parameters(), lr=cfg.actor_lr)
        self.critic_opt = torch.optim.Adam(
            list(self.q1.parameters()) + list(self.q2.parameters()), lr=cfg.critic_lr
        )

        # Automatic temperature tuning
        #self.target_entropy = -float(act_dim)
        self.target_entropy = -0.5 * float(act_dim)  # = -1.0, more exploration
        self.log_alpha  = torch.tensor(0.0, requires_grad=True, device=self.device)
        self.alpha_opt  = torch.optim.Adam([self.log_alpha], lr=cfg.alpha_lr)

        self.act_limit = act_limit

    @property
    def alpha(self):
        return self.log_alpha.exp()

    @torch.no_grad()
    def act(self, obs: np.ndarray, deterministic: bool = False) -> np.ndarray:
        obs_t = to_tensor(obs, self.device).unsqueeze(0)
        a, _, mu_a = self.actor.sample(obs_t)
        out = mu_a if deterministic else a
        return out.squeeze(0).cpu().numpy()

    def _soft_update(self, net, targ):
        tau = self.cfg.tau
        for p, p_t in zip(net.parameters(), targ.parameters()):
            p_t.data.mul_(1 - tau)
            p_t.data.add_(tau * p.data)

    def update(self, batch: Dict[str, torch.Tensor]) -> Dict[str, float]:
        obs      = batch["obs"]
        act      = batch["acts"]
        rew      = batch["rews"]
        next_obs = batch["next_obs"]
        done     = batch["done"]

        # Critic Update
        with torch.no_grad():
            next_a, next_logp, _ = self.actor.sample(next_obs)
            q_next = torch.min(self.q1_t(next_obs, next_a), self.q2_t(next_obs, next_a))
            y = rew + self.cfg.gamma * (1 - done) * (q_next - self.alpha * next_logp)

        q1_val = self.q1(obs, act)
        q2_val = self.q2(obs, act)
```

```
        q_loss = F.mse_loss(q1_val, y) + F.mse_loss(q2_val, y)

        self.critic_opt.zero_grad()
        q_loss.backward()
        self.critic_opt.step()

        # Actor update
        a_pi, logp_pi, _ = self.actor.sample(obs)
        q_pi  = torch.min(self.q1(obs, a_pi), self.q2(obs, a_pi))
        pi_loss = (self.alpha * logp_pi - q_pi).mean()

        self.actor_opt.zero_grad()
        pi_loss.backward()
        #nn.utils.clip_grad_norm_(self.actor.parameters(), max_norm=1)
        self.actor_opt.step()

        # Temp adjustment (optional)
        alpha_loss = -(self.alpha * (logp_pi + self.target_entropy).detach()).mean()
        self.alpha_opt.zero_grad()
        alpha_loss.backward()
        self.alpha_opt.step()

        self._soft_update(self.q1, self.q1_t)
        self._soft_update(self.q2, self.q2_t)

        return {
            "q_loss":     float(q_loss.item()),
            "pi_loss":    float(pi_loss.item()),
            "alpha":      float(self.alpha.item()),
            "alpha_loss": float(alpha_loss.item()),
            "logp_pi":    float(logp_pi.mean().item()),
            "q1_mean":    float(q1_val.mean().item()),
        }


def train(env, cfg: SACConfig):
    set_seed(cfg.seed)


    obs_dim   = env.observation_space.shape[0]
    act_dim   = env.action_space.shape[0]
    act_limit = float(env.action_space.high[0])

    print(f"device␣:␣{cfg.device}")
    print(f"obs_dim:␣{obs_dim}␣␣␣act_dim:␣{act_dim}␣␣␣act_limit:␣{act_limit}")

    agent = SACAgent(obs_dim, act_dim, act_limit, cfg)
    rb    = ReplayBuffer(obs_dim, act_dim, cfg.replay_size, agent.device)

    metrics = {key: [] for key in [
        "episode_returns",
        "mean_rewards",
        "update_steps",
        "q_loss", "pi_loss",
        "alpha", "alpha_loss",
        "logp_pi", "q1_mean",
    ]}

```

```
    obs, _ = env.reset(seed=cfg.seed)
    ep_ret, ep_len = 0.0, 0
    ep_num = 0

    training_start = time.time()

    for t in range(1, cfg.total_steps + 1):

        if t < cfg.start_steps:
            act = env.action_space.sample()
        else:
            act = agent.act(obs, deterministic=False)

        next_obs, rew, terminated, truncated, info = env.step(act)
        done_flag = bool(terminated or truncated)
        done = float(done_flag)

        rb.add(obs, act, rew, next_obs, done)

        obs     = next_obs
        ep_ret += rew
        ep_len += 1

        if done_flag:
            ep_num += 1
            metrics["episode_returns"].append((t, ep_ret))
            metrics["mean_rewards"].append(ep_ret / max(ep_len, 1))
            recent      = metrics["episode_returns"][-10:]
            recent_avg  = np.mean([r for _, r in recent])
            sps         = int(t / (time.time() - training_start))
            print(
                f"[train] step={t:>7d}  ep={ep_num:>4d}  "
                f"return={ep_ret:>8.1f}  len={ep_len:>4d}  "
                f"avg10={recent_avg:>7.1f}  sps={sps}"
            )
            obs, _ = env.reset()
            ep_ret, ep_len = 0.0, 0


        if t >= cfg.update_after and t % cfg.update_every == 0:
            info_list = {k: [] for k in ["q_loss", "pi_loss", "alpha", "alpha_loss", "
                logp_pi", "q1_mean"]}
            for _ in range(cfg.update_every):
                batch = rb.sample(cfg.batch_size)
                info  = agent.update(batch)
                for k in info_list:
                    info_list[k].append(info[k])

            metrics["update_steps"].append(t)
            for k in info_list:
                metrics[k].append(float(np.mean(info_list[k])))


    env.close()
    return metrics, agent
```

Anexos Pregunta 3

p3_main.py

```python
from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch

from ddpg import DDPG
from practice_env import make_env, make_person_params
from utils.noise import OrnsteinUhlenbeckActionNoise
from utils.replay_memory import ReplayMemory, Transition


def load_workouts(path: Path) -> list[dict]:
    with path.open("r", encoding="utf-8") as f:
        data = json.load(f)
    workouts = data["workouts"]
    return sorted(workouts, key=lambda w: float(w["level"]))


def workout_action(raw_action: np.ndarray, obs: np.ndarray, workouts: list[dict], env)
    -> tuple[np.ndarray, dict]:
    # obs[3] is normalized benchmark proxy in measurement_mode="physiology".
    current_level = float(np.clip(obs[3], 0.0, 1.0))
    unlocked = [w for w in workouts if float(w["level"]) <= current_level]
    candidates = unlocked if unlocked else [workouts[0]]

    target = float(raw_action[0])
    selected = min(candidates, key=lambda w: abs(float(w["intensity_score"]) - target))
    action = np.array([float(selected["intensity_score"])], dtype=np.float32)
    action = np.clip(action, env.action_space.low, env.action_space.high).astype(np.
        float32)
    return action, selected


def get_recent_type_counts(actions: list[float], workouts: list[dict], window: int = 7)
    -> dict[str, int]:
    if not actions or not workouts:
        return {}
    type_counts: dict[str, int] = {}
    for action_value in actions[-window:]:
        selected_workout = min(workouts, key=lambda w: abs(float(w["intensity_score"]) -
            float(action_value)))
        workout_type = selected_workout.get("type")
        if workout_type is not None:
            workout_type = str(workout_type)
            type_counts[workout_type] = type_counts.get(workout_type, 0) + 1

    return type_counts


def get_diversidad(actions, obs, workouts):
```

```
    if not actions or not workouts:
        return 0

    return len(get_recent_type_counts(actions, workouts))

def reward(actions, obs, performance, prev_benchmark_score, base_hr, base_spo2, workouts
    , omnisciente):
    dia, resting_hr, spo2, benchmark_score, w_ratio = obs
    P_t = performance[-1]
    P_t_1 = performance[-2] if len(performance) > 1 else 0
    B_t = benchmark_score
    B_t_1 = prev_benchmark_score

    D_t = get_diversidad(actions, obs, workouts)
    H_t = sum(1 for a in actions[-7:] if a > 2.5)

    repeat_penalty = 0.0
    type_concentration_penalty = 0.0
    consecutive_repeat_penalty = 0.0

    if actions and workouts:
        recent = actions[-7:]
        id_counts: dict[str, int] = {}
        recent_ids: list[str] = []
        type_counts = get_recent_type_counts(actions, workouts)
        for a in recent:
            selected = min(workouts, key=lambda w: abs(float(w["intensity_score"]) -
                float(a)))
            wid = selected.get("id")
            if wid is not None:
                recent_ids.append(str(wid))
                id_counts[str(wid)] = id_counts.get(str(wid), 0) + 1

        if id_counts:
            max_count = max(id_counts.values())
            repeat_penalty = float(max(0, max_count - 1))

        if type_counts:
            max_type_count = max(type_counts.values())
            type_concentration_penalty = float(max(0, max_type_count - 1))

        if len(recent_ids) > 1:
            consecutive_repeat_penalty = float(sum(1 for i in range(1, len(recent_ids))
                if recent_ids[i] == recent_ids[i - 1]))

    if omnisciente:
        d_P = P_t - P_t_1 # derivada
        i_P = float(np.sum(np.asarray(performance, dtype=np.float32))) # integral (
            acumulado) para mejorar largo plazo
        # y evitar loops de subebaja q maximizan derivada
        r_t = d_P + 3.0 * i_P
        r_t -= max(0, H_t - 5)
        r_t += 2.0 * D_t
        r_t -= 4.0 * repeat_penalty
        r_t -= 3.0 * type_concentration_penalty
        r_t -= 6.0 * consecutive_repeat_penalty

        return r_t

```

```
    else:
        r_t = (B_t - B_t_1)
        r_t -= max(0, H_t - 5)
        r_t -= 0.1 * abs(actions[-1] - actions[-2]) if len(actions) > 1 else 0
        r_t -= 0.2 * (max(0, resting_hr - base_hr) + max(0, base_spo2 - spo2))
        r_t += 2.0 * D_t
        r_t -= 4.0 * repeat_penalty
        r_t -= 3.0 * type_concentration_penalty
        r_t -= 6.0 * consecutive_repeat_penalty

        return r_t


def running_baseline(values: list[float], window: int) -> float:
    if not values:
        return 0.0
    if window <= 0 or len(values) <= window:
        return float(np.mean(values))
    return float(np.mean(values[-window:]))


def rollout_greedy_policy(env, agent, workouts, device, seed: int | None = None):
    state, _ = env.reset(seed=seed)
    state_t = torch.from_numpy(np.expand_dims(np.asarray(state, dtype=np.float32), axis
        =0)).to(device)

    times: list[int] = []
    chosen_intensity: list[float] = []
    selected_types: list[str] = []
    resting_hr: list[float] = []
    spo2: list[float] = []
    benchmark_score: list[float] = []
    performance: list[float] = []
    benchmark_available: list[float] = []
    intensity_diff: list[float] = []

    done = False
    step = 0
    while not done:
        raw_action = agent.calc_action(state_t, action_noise=None).detach().cpu().numpy
            ()[0]
        action, selected = workout_action(raw_action, state, workouts, env)
        next_state, _, terminated, truncated, _ = env.step(action)
        done = bool(terminated or truncated)

        times.append(step)
        chosen_intensity.append(float(action[0]))
        resting_hr.append(float(next_state[1]))
        spo2.append(float(next_state[2]))
        benchmark_score.append(float(next_state[3]))
        performance.append(float(env.true_performance))
        benchmark_available.append(float(env.benchmark_available))
        selected_types.append(str(selected.get("type", "")))
        if len(chosen_intensity) > 1:
            intensity_diff.append(abs(float(action[0]) - chosen_intensity[-2]))
        else:
            intensity_diff.append(0.0)

        state = np.asarray(next_state, dtype=np.float32)
```

```
        state_t = torch.from_numpy(np.expand_dims(state, axis=0)).to(device)
        step += 1

    return {
        "t": times,
        "intensity": chosen_intensity,
        "selected_types": selected_types,
        "intensity_diff": intensity_diff,
        "resting_hr": resting_hr,
        "spo2": spo2,
        "benchmark_score": benchmark_score,
        "performance": performance,
        "benchmark_available": benchmark_available,
    }


def train_single(device, recovery_speed: str, recovery_tag: str, omnisciente: bool, suffix: str) -> None:
    person_mode = "random_once"
    seed = 0
    params = make_person_params(
        person_mode=person_mode,
        seed=seed,
        athletic_level="low",
        recovery_speed=recovery_speed,
        fitness_retention="high",
    )

    env = make_env(params=params, seed=seed, horizon=365, reward_mode="measured_delta")

    root = Path(__file__).resolve().parent
    workouts = load_workouts(root / "rl_workouts_500.json")

    torch.manual_seed(seed)
    np.random.seed(seed)

    gamma = 0.99
    tau = 0.001
    hidden_size = (400, 300)
    replay_size = 100000
    batch_size = 128
    warmup = 1000
    train_steps = 25000

    agent = DDPG(
        gamma=gamma,
        tau=tau,
        hidden_size=hidden_size,
        num_inputs=env.observation_space.shape[0],
        action_space=env.action_space,
        checkpoint_dir=str(root / "results" / "ddpg_practice"),
    )
    memory = ReplayMemory(replay_size)
    ou_noise = OrnsteinUhlenbeckActionNoise(
        mu=np.zeros(env.action_space.shape[-1], dtype=np.float32),
        sigma=0.2 * np.ones(env.action_space.shape[-1], dtype=np.float32),
    )

    state, _ = env.reset(seed=seed)
```

```
    state_t = torch.from_numpy(np.expand_dims(np.asarray(state, dtype=np.float32), axis=0)).to(device)
    base_hr = float(state[1])
    base_spo2 = float(state[2])

    episode_return = 0.0
    episode_returns: list[float] = []
    episode_mean_abs_performance: list[float] = []
    episode_mean_intensity: list[float] = []
    critic_losses: list[float] = []
    actor_losses: list[float] = []
    used_workouts: list[str] = []
    actions_history: list[float] = []
    performance_history: list[float] = [float(env.true_performance)]
    prev_benchmark_score = float(env.last_benchmark_score)
    hr_history: list[float] = [float(state[1])]
    spo2_history: list[float] = [float(state[2])]
    baseline_window = 100
    episode_abs_performance_history: list[float] = []
    episode_intensity_history: list[float] = []

    for step in range(1, train_steps + 1):
        if step <= warmup:
            action = env.action_space.sample().astype(np.float32)
            chosen = {"id": "warmup_random"}
        else:
            raw_action = agent.calc_action(state_t, ou_noise).detach().cpu().numpy()[0]
            action, chosen = workout_action(raw_action, state, workouts, env)

        next_state, env_reward, terminated, truncated, _ = env.step(action)
        done = bool(terminated or truncated)

        base_hr = running_baseline(hr_history, baseline_window)
        base_spo2 = running_baseline(spo2_history, baseline_window)
        actions_history.append(float(action[0]))
        performance_history.append(float(env.true_performance))
        episode_abs_performance_history.append(abs(float(env.true_performance)))
        episode_intensity_history.append(abs(float(action[0])))
        custom_reward = reward(actions_history, next_state, performance_history, prev_benchmark_score, base_hr, base_spo2, workouts, omnisciente)
        prev_benchmark_score = float(env.last_benchmark_score)
        hr_history.append(float(next_state[1]))
        spo2_history.append(float(next_state[2]))

        action_t = torch.from_numpy(np.expand_dims(np.asarray(action, dtype=np.float32), axis=0)).to(device)
        done_t = torch.tensor([float(done)], dtype=torch.float32, device=device)
        reward_t = torch.tensor([float(custom_reward)], dtype=torch.float32, device=device)
        next_state_t = torch.from_numpy(np.expand_dims(np.asarray(next_state, dtype=np.float32), axis=0)).to(device)

        memory.push(state_t, action_t, done_t, next_state_t, reward_t)

        if len(memory) >= batch_size and step > warmup:
            transitions = memory.sample(batch_size)
            batch = Transition(*zip(*transitions))
            value_loss, policy_loss = agent.update_params(batch)
            critic_losses.append(value_loss)
```

```
            actor_losses.append(policy_loss)

        state = np.asarray(next_state, dtype=np.float32)
        state_t = next_state_t
        episode_return += float(custom_reward)
        used_workouts.append(chosen["id"])

        if done:
            episode_returns.append(episode_return)
            episode_mean_abs_performance.append(float(np.mean(
                episode_abs_performance_history)) if episode_abs_performance_history
                else 0.0)
            episode_mean_intensity.append(float(np.mean(episode_intensity_history)) if
                episode_intensity_history else 0.0)
            episode_return = 0.0
            ou_noise.reset()
            state, _ = env.reset()
            state_t = torch.from_numpy(np.expand_dims(np.asarray(state, dtype=np.float32
                ), axis=0)).to(device)
            actions_history = []
            performance_history = [float(env.true_performance)]
            prev_benchmark_score = float(env.last_benchmark_score)
            hr_history = [float(state[1])]
            spo2_history = [float(state[2])]
            episode_abs_performance_history = []
            episode_intensity_history = []

        if step % 5000 == 0:
            print(
                f"step={step} episodes={len(episode_returns)} "
                f"mean_last10={np.mean(episode_returns[-10:]) if episode_returns else
                    0.0:.3f}"
            )

    agent.save_checkpoint(train_steps, memory)

    rollout = rollout_greedy_policy(env, agent, workouts, device, seed=seed)

    fig, axes = plt.subplots(6, 1, figsize=(12, 14), sharex=True)
    axes[0].plot(rollout["t"], rollout["intensity"], color="tab:orange")
    axes[0].set_title("Greedy Policy: Workout Intensity")
    axes[0].set_ylabel("intensity")
    axes[0].grid(True)
    selected_types = rollout.get("selected_types", None) # plot categories elegidas
    if selected_types:
        preferred_order = ["G", "C", "W"]
        present = [t for t in preferred_order if t in set(selected_types)]
        if not present:
            present = list(dict.fromkeys(selected_types))
        mapping = {t: i for i, t in enumerate(present)}
        type_codes = [mapping.get(t, np.nan) for t in selected_types]
        ax2 = axes[0].twinx()
        ax2.plot(rollout["t"], type_codes, color="tab:gray", linestyle="--", alpha=0.9)
        ax2.set_ylabel("workout type")
        ticks = list(mapping.values())
        labels = list(mapping.keys())
        ax2.set_yticks(ticks)
        ax2.set_yticklabels(labels)
        ax2.set_ylim(-0.5, max(ticks) + 0.5)
```

```
329
330     axes[1].plot(rollout["t"], rollout["resting_hr"], color="tab:red")
331     axes[1].set_title("Measured Resting HR")
332     axes[1].set_ylabel("HR")
333     axes[1].grid(True)
334
335     axes[2].plot(rollout["t"], rollout["spo2"], color="tab:blue")
336     axes[2].set_title("Measured SpO2")
337     axes[2].set_ylabel("SpO2")
338     axes[2].grid(True)
339
340     axes[3].plot(rollout["t"], rollout["benchmark_score"], color="tab:green")
341     axes[3].set_title("Measured Benchmark Score")
342     axes[3].set_ylabel("benchmark")
343     axes[3].grid(True)
344
345     axes[4].plot(rollout["t"], rollout["performance"], color="tab:purple")
346     axes[4].set_title("True Performance")
347     axes[4].set_ylabel("performance")
348     axes[4].grid(True)
349
350     axes[5].plot(rollout["t"], rollout["intensity_diff"], color="tab:brown")
351     axes[5].set_title("|w_t - w_t-1|")
352     axes[5].set_xlabel("Time step")
353     axes[5].set_ylabel("intensity change")
354     axes[5].grid(True)
355
356     plt.tight_layout()
357     plt.savefig(root / "results" / "ddpg_practice" / f"policy_rollout_{suffix}.png", dpi
        =150)
358     plt.close()
359
360     fig, axes = plt.subplots(3, 1, figsize=(11, 10), sharex=True)
361     axes[0].plot(episode_returns, color="tab:blue")
362     axes[0].set_title("DDPG Episode Return")
363     axes[0].set_ylabel("Return")
364     axes[0].grid(True)
365
366     axes[1].plot(episode_mean_abs_performance, color="tab:green")
367     axes[1].set_title("Mean Absolute Performance per Episode")
368     axes[1].set_ylabel("|performance|")
369     axes[1].grid(True)
370
371     axes[2].plot(episode_mean_intensity, color="tab:orange")
372     axes[2].set_title("Mean Workout Intensity per Episode")
373     axes[2].set_xlabel("Episode")
374     axes[2].set_ylabel("intensity")
375     axes[2].grid(True)
376
377     plt.tight_layout()
378     plt.savefig(root / "results" / "ddpg_practice" / f"episode_metrics_{suffix}.png",
        dpi=150)
379     plt.close()
380
381     if actor_losses and critic_losses:
382         plt.figure(figsize=(10, 5))
383         plt.plot(actor_losses, label="actor_loss", alpha=0.7)
384         plt.plot(critic_losses, label="critic_loss", alpha=0.7)
385         plt.title("DDPG Losses")
```

```
        plt.xlabel("Update")
        plt.ylabel("Loss")
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.savefig(root / "results" / "ddpg_practice" / f"losses_{suffix}.png", dpi
            =150)
        plt.close()

    unique, counts = np.unique(np.asarray(used_workouts, dtype=object), return_counts=
        True)
    ranking = sorted(zip(unique.tolist(), counts.tolist()), key=lambda x: x[1], reverse=
        True)[:10]
    print(f"\n[{suffix}] Top workouts usados:")
    for wid, n in ranking:
        print(f"  {wid}: {n}")


def main() -> None:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    recovery_configs = [("high", "high"), ("med", "medium"), ("low", "low")]
    omnisciente_options = [False, True]
    iteration = 1
    total = len(recovery_configs) * len(omnisciente_options)

    for recovery_tag, recovery_speed in recovery_configs:
        for omni in omnisciente_options:
            suffix = recovery_tag
            if omni:
                suffix += "_omni"

            print(f"\n
                {'================================================================='}
                ")
            print(f"Iteration {iteration}/{total}: recovery={recovery_tag}, omnisciente
                ={omni}")
            train_single(device, recovery_speed, recovery_tag, omni, suffix)
            iteration += 1

if __name__ == "__main__":
    main()
```

```
# DDPG de github.com/songrotek/DDPG/blob/master/ddpg.py

import gc
import logging
import os

import numpy as np
import torch
import torch.nn as nn

import torch.nn.functional as F
from torch.optim import Adam


def fanin_init(size, fanin=None):
    fanin = fanin or size[0]
```

```
    v = 1.0 / np.sqrt(fanin)
    return torch.Tensor(size).uniform_(-v, v)


class Actor(nn.Module):
    def __init__(self, hidden_size, num_inputs, action_space, init_w=3e-3):
        super().__init__()
        hidden1, hidden2 = int(hidden_size[0]), int(hidden_size[1])
        self.fc1 = nn.Linear(num_inputs, hidden1)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, int(action_space.shape[0]))
        self.relu = nn.ReLU()
        self.tanh = nn.Tanh()
        self.action_low = torch.as_tensor(action_space.low, dtype=torch.float32)
        self.action_high = torch.as_tensor(action_space.high, dtype=torch.float32)
        self.action_scale = (self.action_high - self.action_low) / 2.0
        self.action_bias = (self.action_high + self.action_low) / 2.0
        self.init_weights(init_w)

    def init_weights(self, init_w):
        self.fc1.weight.data = fanin_init(self.fc1.weight.data.size())
        self.fc2.weight.data = fanin_init(self.fc2.weight.data.size())
        self.fc3.weight.data.uniform_(-init_w, init_w)

    def forward(self, x):
        out = self.relu(self.fc1(x))
        out = self.relu(self.fc2(out))
        out = self.tanh(self.fc3(out))
        return self.action_bias.to(out.device) + self.action_scale.to(out.device) * out


class Critic(nn.Module):
    def __init__(self, hidden_size, num_inputs, action_space, init_w=3e-3):
        super().__init__()
        hidden1, hidden2 = int(hidden_size[0]), int(hidden_size[1])
        n_actions = int(action_space.shape[0])
        self.fc1 = nn.Linear(num_inputs, hidden1)
        self.fc2 = nn.Linear(hidden1 + n_actions, hidden2)
        self.fc3 = nn.Linear(hidden2, 1)
        self.relu = nn.ReLU()
        self.init_weights(init_w)

    def init_weights(self, init_w):
        self.fc1.weight.data = fanin_init(self.fc1.weight.data.size())
        self.fc2.weight.data = fanin_init(self.fc2.weight.data.size())
        self.fc3.weight.data.uniform_(-init_w, init_w)

    def forward(self, state, action):
        out = self.relu(self.fc1(state))
        out = self.relu(self.fc2(torch.cat([out, action], dim=1)))
        out = self.fc3(out)
        return out

logger = logging.getLogger('ddpg')
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

# if gpu is to be used
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
76
77
78  def soft_update(target, source, tau):
79      for target_param, param in zip(target.parameters(), source.parameters()):
80          target_param.data.copy_(target_param.data * (1.0 - tau) + param.data * tau)
81
82
83  def hard_update(target, source):
84      for target_param, param in zip(target.parameters(), source.parameters()):
85          target_param.data.copy_(param.data)
86
87
88  class DDPG(object):
89
90      def __init__(self, gamma, tau, hidden_size, num_inputs, action_space, checkpoint_dir
            =None):
91          """
92  ␣␣␣␣␣␣␣␣Deep␣Deterministic␣Policy␣Gradient
93  ␣␣␣␣␣␣␣␣Read␣the␣detail␣about␣it␣here:
94  ␣␣␣␣␣␣␣␣https://arxiv.org/abs/1509.02971
95
96  ␣␣␣␣␣␣␣␣Arguments:
97  ␣␣␣␣␣␣␣␣␣␣␣␣gamma:␣␣␣␣␣␣␣␣␣␣␣Discount␣factor
98  ␣␣␣␣␣␣␣␣␣␣␣␣tau:␣␣␣␣␣␣␣␣␣␣␣␣Update␣factor␣for␣the␣actor␣and␣the␣critic
99  ␣␣␣␣␣␣␣␣␣␣␣␣hidden_size:␣␣␣␣Number␣of␣units␣in␣the␣hidden␣layers␣of␣the␣actor␣and␣critic
        .␣Must␣be␣of␣length␣2.
100 ␣␣␣␣␣␣␣␣␣␣␣␣num_inputs:␣␣␣␣␣Size␣of␣the␣input␣states
101 ␣␣␣␣␣␣␣␣␣␣␣␣action_space:␣␣␣The␣action␣space␣of␣the␣used␣environment.␣Used␣to␣clip␣the␣
        actions␣and
102 ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣to␣distinguish␣the␣number␣of␣outputs
103 ␣␣␣␣␣␣␣␣␣␣␣␣checkpoint_dir:␣Path␣as␣String␣to␣the␣directory␣to␣save␣the␣networks.
104 ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣If␣None␣then␣"./saved_models/"␣will␣be␣used
105 ␣␣␣␣␣␣␣␣"""
106
107         self.gamma = gamma
108         self.tau = tau
109         self.action_space = action_space
110
111         # Define the actor
112         self.actor = Actor(hidden_size, num_inputs, self.action_space).to(device)
113         self.actor_target = Actor(hidden_size, num_inputs, self.action_space).to(device)
114
115         # Define the critic
116         self.critic = Critic(hidden_size, num_inputs, self.action_space).to(device)
117         self.critic_target = Critic(hidden_size, num_inputs, self.action_space).to(
                device)
118
119         # Define the optimizers for both networks
120         self.actor_optimizer = Adam(self.actor.parameters(),
121                                     lr=1e-4)  # optimizer for the actor network
122         self.critic_optimizer = Adam(self.critic.parameters(),
123                                      lr=1e-3,
124                                      weight_decay=1e-2
125                                      )  # optimizer for the critic network
126
127         # Make sure both targets are with the same weight
128         hard_update(self.actor_target, self.actor)
129         hard_update(self.critic_target, self.critic)
130
```

```
131         # Set the directory to save the models
132         if checkpoint_dir is None:
133             self.checkpoint_dir = "./saved_models/"
134         else:
135             self.checkpoint_dir = checkpoint_dir
136         os.makedirs(self.checkpoint_dir, exist_ok=True)
137         logger.info('Saving␣all␣checkpoints␣to␣{}'.format(self.checkpoint_dir))
138 
139     def calc_action(self, state, action_noise=None):
140         """
141 ␣␣␣␣␣␣␣␣Evaluates␣the␣action␣to␣perform␣in␣a␣given␣state
142 
143 ␣␣␣␣␣␣␣␣Arguments:
144 ␣␣␣␣␣␣␣␣␣␣␣␣state:␣␣␣␣␣␣␣␣␣␣␣State␣to␣perform␣the␣action␣on␣in␣the␣env.
145 ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Used␣to␣evaluate␣the␣action.
146 ␣␣␣␣␣␣␣␣␣␣␣␣action_noise:␣␣␣If␣not␣None,␣the␣noise␣to␣apply␣on␣the␣evaluated␣action
147 ␣␣␣␣␣␣␣␣"""
148         x = state.to(device)
149 
150         # Get the continous action value to perform in the env
151         self.actor.eval()  # Sets the actor in evaluation mode
152         mu = self.actor(x)
153         self.actor.train()  # Sets the actor in training mode
154         mu = mu.data
155 
156         # During training we add noise for exploration
157         if action_noise is not None:
158             noise = torch.Tensor(action_noise.noise()).to(device)
159             mu += noise
160 
161         # Clip the output according to the action space of the env
162         mu = mu.clamp(self.action_space.low[0], self.action_space.high[0])
163 
164         return mu
165 
166     def update_params(self, batch):
167         """
168 ␣␣␣␣␣␣␣␣Updates␣the␣parameters/networks␣of␣the␣agent␣according␣to␣the␣given␣batch.
169 ␣␣␣␣␣␣␣␣This␣means␣we␣...
170 ␣␣␣␣␣␣␣␣␣␣␣␣1.␣Compute␣the␣targets
171 ␣␣␣␣␣␣␣␣␣␣␣␣2.␣Update␣the␣Q-function/critic␣by␣one␣step␣of␣gradient␣descent
172 ␣␣␣␣␣␣␣␣␣␣␣␣3.␣Update␣the␣policy/actor␣by␣one␣step␣of␣gradient␣ascent
173 ␣␣␣␣␣␣␣␣␣␣␣␣4.␣Update␣the␣target␣networks␣through␣a␣soft␣update
174 
175 ␣␣␣␣␣␣␣␣Arguments:
176 ␣␣␣␣␣␣␣␣␣␣␣␣batch:␣␣Batch␣to␣perform␣the␣training␣of␣the␣parameters
177 ␣␣␣␣␣␣␣␣"""
178         # Get tensors from the batch
179         state_batch = torch.cat(batch.state).to(device)
180         action_batch = torch.cat(batch.action).to(device)
181         reward_batch = torch.cat(batch.reward).to(device)
182         done_batch = torch.cat(batch.done).to(device)
183         next_state_batch = torch.cat(batch.next_state).to(device)
184 
185         # Get the actions and the state values to compute the targets
186         next_action_batch = self.actor_target(next_state_batch)
187         next_state_action_values = self.critic_target(next_state_batch,
                next_action_batch.detach())
188 
```

```
189         # Compute the target
190         reward_batch = reward_batch.unsqueeze(1)
191         done_batch = done_batch.unsqueeze(1)
192         expected_values = reward_batch + (1.0 - done_batch) * self.gamma *
                next_state_action_values
193
194         # TODO: Clipping the expected values here?
195         # expected_value = torch.clamp(expected_value, min_value, max_value)
196
197         # Update the critic network
198         self.critic_optimizer.zero_grad()
199         state_action_batch = self.critic(state_batch, action_batch)
200         value_loss = F.mse_loss(state_action_batch, expected_values.detach())
201         value_loss.backward()
202         self.critic_optimizer.step()
203
204         # Update the actor network
205         self.actor_optimizer.zero_grad()
206         policy_loss = -self.critic(state_batch, self.actor(state_batch))
207         policy_loss = policy_loss.mean()
208         policy_loss.backward()
209         self.actor_optimizer.step()
210
211         # Update the target networks
212         soft_update(self.actor_target, self.actor, self.tau)
213         soft_update(self.critic_target, self.critic, self.tau)
214
215         return value_loss.item(), policy_loss.item()
216
217     def save_checkpoint(self, last_timestep, replay_buffer):
218         """
219 ␣␣␣␣␣␣␣␣Saving␣the␣networks␣and␣all␣parameters␣to␣a␣file␣in␣'checkpoint_dir'
220
221 ␣␣␣␣␣␣␣␣Arguments:
222 ␣␣␣␣␣␣␣␣␣␣␣␣last_timestep:␣␣Last␣timestep␣in␣training␣before␣saving
223 ␣␣␣␣␣␣␣␣␣␣␣␣replay_buffer:␣␣Current␣replay␣buffer
224 ␣␣␣␣␣␣␣␣"""
225         checkpoint_name = self.checkpoint_dir + '/ep_{}.pth.tar'.format(last_timestep)
226         logger.info('Saving␣checkpoint...')
227         checkpoint = {
228             'last_timestep': last_timestep,
229             'actor': self.actor.state_dict(),
230             'critic': self.critic.state_dict(),
231             'actor_target': self.actor_target.state_dict(),
232             'critic_target': self.critic_target.state_dict(),
233             'actor_optimizer': self.actor_optimizer.state_dict(),
234             'critic_optimizer': self.critic_optimizer.state_dict(),
235             'replay_buffer': replay_buffer,
236         }
237         logger.info('Saving␣model␣at␣timestep␣{}...'.format(last_timestep))
238         torch.save(checkpoint, checkpoint_name)
239         gc.collect()
240         logger.info('Saved␣model␣at␣timestep␣{}␣to␣{}'.format(last_timestep, self.
                checkpoint_dir))
241
242     def get_path_of_latest_file(self):
243         """
244 ␣␣␣␣␣␣␣␣Returns␣the␣latest␣created␣file␣in␣'checkpoint_dir'
245 ␣␣␣␣␣␣␣␣"""
```

```
246         files = [file for file in os.listdir(self.checkpoint_dir) if (file.endswith(".pt
                ") or file.endswith(".tar"))]
247         filepaths = [os.path.join(self.checkpoint_dir, file) for file in files]
248         last_file = max(filepaths, key=os.path.getctime)
249         return os.path.abspath(last_file)
250 
251     def load_checkpoint(self, checkpoint_path=None):
252         """
253 ␣␣␣␣␣␣␣␣Saving␣the␣networks␣and␣all␣parameters␣from␣a␣given␣path.␣If␣the␣given␣path␣is␣
        None
254 ␣␣␣␣␣␣␣␣then␣the␣latest␣saved␣file␣in␣'checkpoint_dir'␣will␣be␣used.
255 
256 ␣␣␣␣␣␣␣␣Arguments:
257 ␣␣␣␣␣␣␣␣␣␣␣␣checkpoint_path:␣␣␣␣File␣to␣load␣the␣model␣from
258 
259 ␣␣␣␣␣␣␣␣"""
260 
261         if checkpoint_path is None:
262             checkpoint_path = self.get_path_of_latest_file()
263 
264         if os.path.isfile(checkpoint_path):
265             logger.info("Loading␣checkpoint...({})".format(checkpoint_path))
266             key = 'cuda' if torch.cuda.is_available() else 'cpu'
267 
268             checkpoint = torch.load(checkpoint_path, map_location=key, weights_only=
                    False)
269             start_timestep = checkpoint['last_timestep'] + 1
270             self.actor.load_state_dict(checkpoint['actor'])
271             self.critic.load_state_dict(checkpoint['critic'])
272             self.actor_target.load_state_dict(checkpoint['actor_target'])
273             self.critic_target.load_state_dict(checkpoint['critic_target'])
274             self.actor_optimizer.load_state_dict(checkpoint['actor_optimizer'])
275             self.critic_optimizer.load_state_dict(checkpoint['critic_optimizer'])
276             replay_buffer = checkpoint['replay_buffer']
277 
278             gc.collect()
279             logger.info('Loaded␣model␣at␣timestep␣{}␣from␣{}'.format(start_timestep,
                    checkpoint_path))
280             return start_timestep, replay_buffer
281         else:
282             raise OSError('Checkpoint␣not␣found')
283 
284     def set_eval(self):
285         """
286 ␣␣␣␣␣␣␣␣Sets␣the␣model␣in␣evaluation␣mode
287 
288 ␣␣␣␣␣␣␣␣"""
289         self.actor.eval()
290         self.critic.eval()
291         self.actor_target.eval()
292         self.critic_target.eval()
293 
294     def set_train(self):
295         """
296 ␣␣␣␣␣␣␣␣Sets␣the␣model␣in␣training␣mode
297 
298 ␣␣␣␣␣␣␣␣"""
299         self.actor.train()
300         self.critic.train()
```

```
        self.actor_target.train()
        self.critic_target.train()

    def get_network(self, name):
        if name == 'Actor':
            return self.actor
        elif name == 'Critic':
            return self.critic
        else:
            raise NameError('name \'{}\' is not defined as a network'.format(name))
```

ddpg.py

```
# DDPG de github.com/songrotek/DDPG/blob/master/ddpg.py

import gc
import logging
import os

import numpy as np
import torch
import torch.nn as nn

import torch.nn.functional as F
from torch.optim import Adam


def fanin_init(size, fanin=None):
    fanin = fanin or size[0]
    v = 1.0 / np.sqrt(fanin)
    return torch.Tensor(size).uniform_(-v, v)


class Actor(nn.Module):
    def __init__(self, hidden_size, num_inputs, action_space, init_w=3e-3):
        super().__init__()
        hidden1, hidden2 = int(hidden_size[0]), int(hidden_size[1])
        self.fc1 = nn.Linear(num_inputs, hidden1)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, int(action_space.shape[0]))
        self.relu = nn.ReLU()
        self.tanh = nn.Tanh()
        self.action_low = torch.as_tensor(action_space.low, dtype=torch.float32)
        self.action_high = torch.as_tensor(action_space.high, dtype=torch.float32)
        self.action_scale = (self.action_high - self.action_low) / 2.0
        self.action_bias = (self.action_high + self.action_low) / 2.0
        self.init_weights(init_w)

    def init_weights(self, init_w):
        self.fc1.weight.data = fanin_init(self.fc1.weight.data.size())
        self.fc2.weight.data = fanin_init(self.fc2.weight.data.size())
        self.fc3.weight.data.uniform_(-init_w, init_w)

    def forward(self, x):
        out = self.relu(self.fc1(x))
        out = self.relu(self.fc2(out))
        out = self.tanh(self.fc3(out))
```

```
45          return self.action_bias.to(out.device) + self.action_scale.to(out.device) * out
46
47
48  class Critic(nn.Module):
49      def __init__(self, hidden_size, num_inputs, action_space, init_w=3e-3):
50          super().__init__()
51          hidden1, hidden2 = int(hidden_size[0]), int(hidden_size[1])
52          n_actions = int(action_space.shape[0])
53          self.fc1 = nn.Linear(num_inputs, hidden1)
54          self.fc2 = nn.Linear(hidden1 + n_actions, hidden2)
55          self.fc3 = nn.Linear(hidden2, 1)
56          self.relu = nn.ReLU()
57          self.init_weights(init_w)
58
59      def init_weights(self, init_w):
60          self.fc1.weight.data = fanin_init(self.fc1.weight.data.size())
61          self.fc2.weight.data = fanin_init(self.fc2.weight.data.size())
62          self.fc3.weight.data.uniform_(-init_w, init_w)
63
64      def forward(self, state, action):
65          out = self.relu(self.fc1(state))
66          out = self.relu(self.fc2(torch.cat([out, action], dim=1)))
67          out = self.fc3(out)
68          return out
69
70  logger = logging.getLogger('ddpg')
71  logger.setLevel(logging.INFO)
72  logger.addHandler(logging.StreamHandler())
73
74  # if gpu is to be used
75  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
76
77
78  def soft_update(target, source, tau):
79      for target_param, param in zip(target.parameters(), source.parameters()):
80          target_param.data.copy_(target_param.data * (1.0 - tau) + param.data * tau)
81
82
83  def hard_update(target, source):
84      for target_param, param in zip(target.parameters(), source.parameters()):
85          target_param.data.copy_(param.data)
86
87
88  class DDPG(object):
89
90      def __init__(self, gamma, tau, hidden_size, num_inputs, action_space, checkpoint_dir
        =None):
91          """
92  ␣␣␣␣␣␣␣␣Deep␣Deterministic␣Policy␣Gradient
93  ␣␣␣␣␣␣␣␣Read␣the␣detail␣about␣it␣here:
94  ␣␣␣␣␣␣␣␣https://arxiv.org/abs/1509.02971
95
96  ␣␣␣␣␣␣␣␣Arguments:
97  ␣␣␣␣␣␣␣␣␣␣␣␣gamma:␣␣␣␣␣␣␣␣␣␣␣Discount␣factor
98  ␣␣␣␣␣␣␣␣␣␣␣␣tau:␣␣␣␣␣␣␣␣␣␣␣␣Update␣factor␣for␣the␣actor␣and␣the␣critic
99  ␣␣␣␣␣␣␣␣␣␣␣␣hidden_size:␣␣␣␣Number␣of␣units␣in␣the␣hidden␣layers␣of␣the␣actor␣and␣critic
    .␣Must␣be␣of␣length␣2.
100 ␣␣␣␣␣␣␣␣␣␣␣␣num_inputs:␣␣␣␣␣Size␣of␣the␣input␣states
```

```
            action_space:   The action space of the used environment. Used to clip the actions and
                            to distinguish the number of outputs
            checkpoint_dir: Path as String to the directory to save the networks.
                            If None then "./saved_models/" will be used
        """

        self.gamma = gamma
        self.tau = tau
        self.action_space = action_space

        # Define the actor
        self.actor = Actor(hidden_size, num_inputs, self.action_space).to(device)
        self.actor_target = Actor(hidden_size, num_inputs, self.action_space).to(device)

        # Define the critic
        self.critic = Critic(hidden_size, num_inputs, self.action_space).to(device)
        self.critic_target = Critic(hidden_size, num_inputs, self.action_space).to(
            device)

        # Define the optimizers for both networks
        self.actor_optimizer = Adam(self.actor.parameters(),
                                    lr=1e-4)  # optimizer for the actor network
        self.critic_optimizer = Adam(self.critic.parameters(),
                                     lr=1e-3,
                                     weight_decay=1e-2
                                     )  # optimizer for the critic network

        # Make sure both targets are with the same weight
        hard_update(self.actor_target, self.actor)
        hard_update(self.critic_target, self.critic)

        # Set the directory to save the models
        if checkpoint_dir is None:
            self.checkpoint_dir = "./saved_models/"
        else:
            self.checkpoint_dir = checkpoint_dir
        os.makedirs(self.checkpoint_dir, exist_ok=True)
        logger.info('Saving all checkpoints to {}'.format(self.checkpoint_dir))

    def calc_action(self, state, action_noise=None):
        """
        Evaluates the action to perform in a given state

        Arguments:
            state:          State to perform the action on in the env.
                            Used to evaluate the action.
            action_noise:   If not None, the noise to apply on the evaluated action
        """
        x = state.to(device)

        # Get the continous action value to perform in the env
        self.actor.eval()  # Sets the actor in evaluation mode
        mu = self.actor(x)
        self.actor.train()  # Sets the actor in training mode
        mu = mu.data

        # During training we add noise for exploration
        if action_noise is not None:
```

```
158             noise = torch.Tensor(action_noise.noise()).to(device)
159             mu += noise
160 
161         # Clip the output according to the action space of the env
162         mu = mu.clamp(self.action_space.low[0], self.action_space.high[0])
163 
164         return mu
165 
166     def update_params(self, batch):
167         """
168 ␣␣␣␣␣␣␣␣Updates␣the␣parameters/networks␣of␣the␣agent␣according␣to␣the␣given␣batch.
169 ␣␣␣␣␣␣␣␣This␣means␣we␣...
170 ␣␣␣␣␣␣␣␣␣␣␣␣1.␣Compute␣the␣targets
171 ␣␣␣␣␣␣␣␣␣␣␣␣2.␣Update␣the␣Q-function/critic␣by␣one␣step␣of␣gradient␣descent
172 ␣␣␣␣␣␣␣␣␣␣␣␣3.␣Update␣the␣policy/actor␣by␣one␣step␣of␣gradient␣ascent
173 ␣␣␣␣␣␣␣␣␣␣␣␣4.␣Update␣the␣target␣networks␣through␣a␣soft␣update
174 
175 ␣␣␣␣␣␣␣␣Arguments:
176 ␣␣␣␣␣␣␣␣␣␣␣␣batch:␣␣Batch␣to␣perform␣the␣training␣of␣the␣parameters
177 ␣␣␣␣␣␣␣␣"""
178         # Get tensors from the batch
179         state_batch = torch.cat(batch.state).to(device)
180         action_batch = torch.cat(batch.action).to(device)
181         reward_batch = torch.cat(batch.reward).to(device)
182         done_batch = torch.cat(batch.done).to(device)
183         next_state_batch = torch.cat(batch.next_state).to(device)
184 
185         # Get the actions and the state values to compute the targets
186         next_action_batch = self.actor_target(next_state_batch)
187         next_state_action_values = self.critic_target(next_state_batch,
                next_action_batch.detach())
188 
189         # Compute the target
190         reward_batch = reward_batch.unsqueeze(1)
191         done_batch = done_batch.unsqueeze(1)
192         expected_values = reward_batch + (1.0 - done_batch) * self.gamma *
                next_state_action_values
193 
194         # TODO: Clipping the expected values here?
195         # expected_value = torch.clamp(expected_value, min_value, max_value)
196 
197         # Update the critic network
198         self.critic_optimizer.zero_grad()
199         state_action_batch = self.critic(state_batch, action_batch)
200         value_loss = F.mse_loss(state_action_batch, expected_values.detach())
201         value_loss.backward()
202         self.critic_optimizer.step()
203 
204         # Update the actor network
205         self.actor_optimizer.zero_grad()
206         policy_loss = -self.critic(state_batch, self.actor(state_batch))
207         policy_loss = policy_loss.mean()
208         policy_loss.backward()
209         self.actor_optimizer.step()
210 
211         # Update the target networks
212         soft_update(self.actor_target, self.actor, self.tau)
213         soft_update(self.critic_target, self.critic, self.tau)
214 
```

```
215         return value_loss.item(), policy_loss.item()
216
217     def save_checkpoint(self, last_timestep, replay_buffer):
218         """
219 ␣␣␣␣␣␣␣␣Saving␣the␣networks␣and␣all␣parameters␣to␣a␣file␣in␣'checkpoint_dir'
220
221 ␣␣␣␣␣␣␣␣Arguments:
222 ␣␣␣␣␣␣␣␣␣␣␣␣last_timestep:␣␣Last␣timestep␣in␣training␣before␣saving
223 ␣␣␣␣␣␣␣␣␣␣␣␣replay_buffer:␣␣Current␣replay␣buffer
224 ␣␣␣␣␣␣␣␣"""
225         checkpoint_name = self.checkpoint_dir + '/ep_{}.pth.tar'.format(last_timestep)
226         logger.info('Saving␣checkpoint...')
227         checkpoint = {
228             'last_timestep': last_timestep,
229             'actor': self.actor.state_dict(),
230             'critic': self.critic.state_dict(),
231             'actor_target': self.actor_target.state_dict(),
232             'critic_target': self.critic_target.state_dict(),
233             'actor_optimizer': self.actor_optimizer.state_dict(),
234             'critic_optimizer': self.critic_optimizer.state_dict(),
235             'replay_buffer': replay_buffer,
236         }
237         logger.info('Saving␣model␣at␣timestep␣{}...'.format(last_timestep))
238         torch.save(checkpoint, checkpoint_name)
239         gc.collect()
240         logger.info('Saved␣model␣at␣timestep␣{}␣to␣{}'.format(last_timestep, self.
                checkpoint_dir))
241
242     def get_path_of_latest_file(self):
243         """
244 ␣␣␣␣␣␣␣␣Returns␣the␣latest␣created␣file␣in␣'checkpoint_dir'
245 ␣␣␣␣␣␣␣␣"""
246         files = [file for file in os.listdir(self.checkpoint_dir) if (file.endswith(".pt
                ") or file.endswith(".tar"))]
247         filepaths = [os.path.join(self.checkpoint_dir, file) for file in files]
248         last_file = max(filepaths, key=os.path.getctime)
249         return os.path.abspath(last_file)
250
251     def load_checkpoint(self, checkpoint_path=None):
252         """
253 ␣␣␣␣␣␣␣␣Saving␣the␣networks␣and␣all␣parameters␣from␣a␣given␣path.␣If␣the␣given␣path␣is␣
        None
254 ␣␣␣␣␣␣␣␣then␣the␣latest␣saved␣file␣in␣'checkpoint_dir'␣will␣be␣used.
255
256 ␣␣␣␣␣␣␣␣Arguments:
257 ␣␣␣␣␣␣␣␣␣␣␣␣checkpoint_path:␣␣␣␣File␣to␣load␣the␣model␣from
258
259 ␣␣␣␣␣␣␣␣"""
260
261         if checkpoint_path is None:
262             checkpoint_path = self.get_path_of_latest_file()
263
264         if os.path.isfile(checkpoint_path):
265             logger.info("Loading␣checkpoint...({})".format(checkpoint_path))
266             key = 'cuda' if torch.cuda.is_available() else 'cpu'
267
268             checkpoint = torch.load(checkpoint_path, map_location=key, weights_only=
                    False)
269             start_timestep = checkpoint['last_timestep'] + 1
```

```
            self.actor.load_state_dict(checkpoint['actor'])
            self.critic.load_state_dict(checkpoint['critic'])
            self.actor_target.load_state_dict(checkpoint['actor_target'])
            self.critic_target.load_state_dict(checkpoint['critic_target'])
            self.actor_optimizer.load_state_dict(checkpoint['actor_optimizer'])
            self.critic_optimizer.load_state_dict(checkpoint['critic_optimizer'])
            replay_buffer = checkpoint['replay_buffer']

            gc.collect()
            logger.info('Loaded␣model␣at␣timestep␣{}␣from␣{}'.format(start_timestep,
                checkpoint_path))
            return start_timestep, replay_buffer
        else:
            raise OSError('Checkpoint␣not␣found')

    def set_eval(self):
        """
␣␣␣␣␣␣␣␣Sets␣the␣model␣in␣evaluation␣mode

␣␣␣␣␣␣␣␣"""
        self.actor.eval()
        self.critic.eval()
        self.actor_target.eval()
        self.critic_target.eval()

    def set_train(self):
        """
␣␣␣␣␣␣␣␣Sets␣the␣model␣in␣training␣mode

␣␣␣␣␣␣␣␣"""
        self.actor.train()
        self.critic.train()
        self.actor_target.train()
        self.critic_target.train()

    def get_network(self, name):
        if name == 'Actor':
            return self.actor
        elif name == 'Critic':
            return self.critic
        else:
            raise NameError('name␣\'{}\'␣is␣not␣defined␣as␣a␣network'.format(name))
```

Referencias

[1] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, and CleanRL Contributors. ppo_continuous_action.py. `https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/ppo_continuous_action.py`, 2026. Accessed: 2026-05-15.

[2] Schneimo. ddpg-pytorch. `https://github.com/schneimo/ddpg-pytorch`, n.d. GitHub repository. Accessed: 2026-05-13.